

Auto-Healing and Scaling Applications

A practical introduction to EC2 Auto Scaling and CloudWatch

Table of Contents

- 00 Auto Scaling Recap**
- 01 Auto-Healing & Scaling**
- 02 Practice Lab**

00 Auto Scaling Recap

Background

Our gaming café is trending.

- Customers host their own game servers.
- There is a daily limit on the number of EC2 instances customers can provision.
- We currently can't enforce this limit automatically.

Problems

- If an EC2 instance crashes, we have no way to replace it automatically.
- We need to enforce our daily server limit for each customer.
- Every Tuesday, we host a large party from 8:00 PM to 1:00 AM.

We need the servers ready when the party starts and shut down automatically when it ends.

In other words: can AWS automatically replace failed game servers, enforce limits, and schedule when capacity starts and stops?

Solution

Auto Scaling groups would be a good fit for this scenario. We can create launch templates for our customers' game servers and configure the group to automatically scale the number of EC2 instances up or down as demand changes.

We can also define maximum limits on how many instances each customer can provision.

With Auto Scaling groups, we can set up scheduled scaling to launch and terminate instances according to a schedule we define.

01 Auto-Healing & Scaling

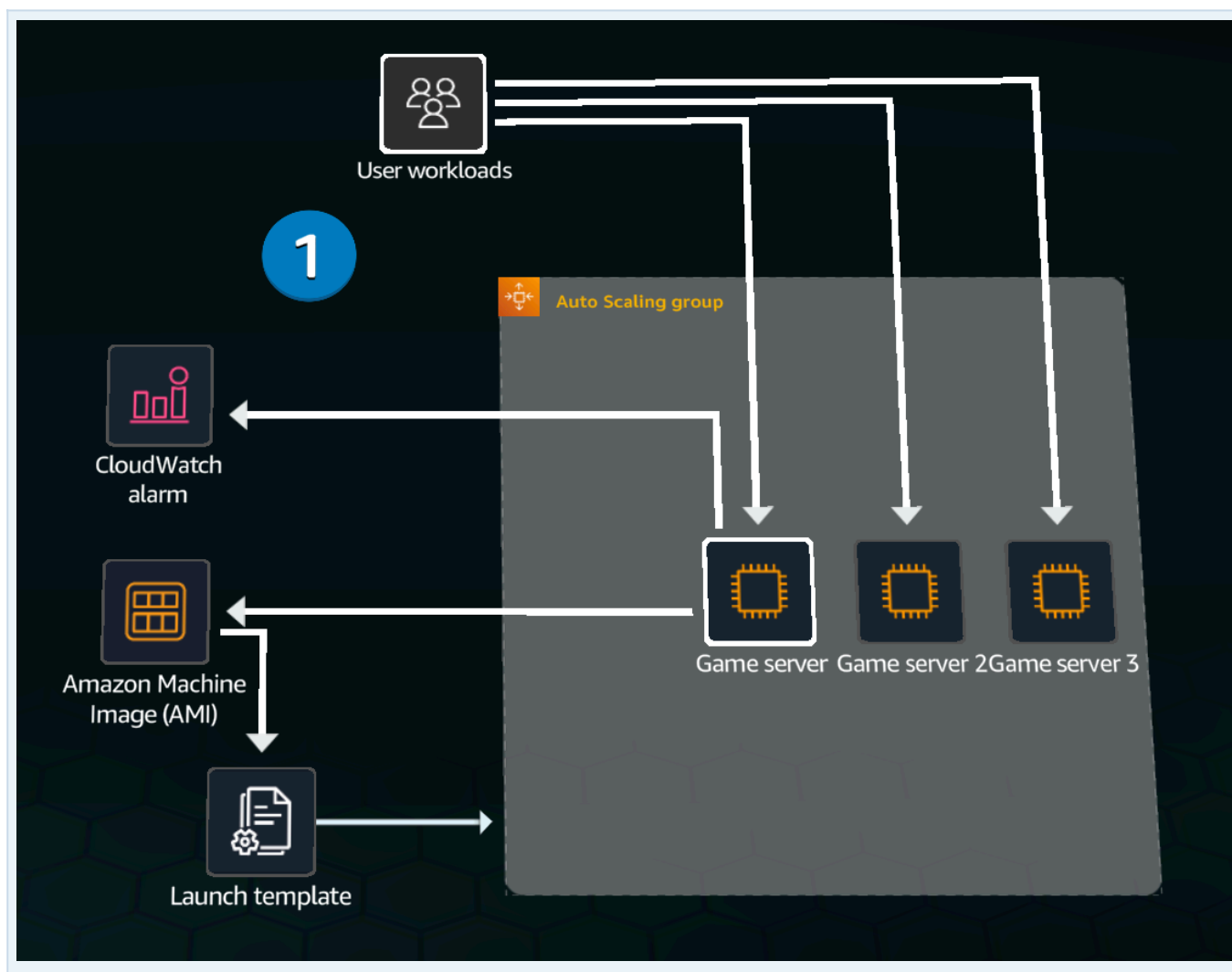
In this solution, an Amazon EC2 Auto Scaling group is created to maintain the optimal number of EC2 instances needed to handle varying application server workloads.

What happens when our EC2 instances can't handle the demand?

Imagine we are running an online game using several EC2 instances as game servers. During normal hours, three servers might be enough. But if thousands of players suddenly join, those servers could become overloaded. If one server fails, we also don't want to manually launch a replacement.

We need AWS to adjust the number of servers as demand changes and replace unhealthy instances automatically.

For now, the focus is on one thing only: setting up a managed group of EC2 instances that AWS can later scale and heal automatically.



Component	Purpose
Auto Scaling group	Manages the EC2 instances
CloudWatch	Monitors metrics and alarms
Launch template	Defines how new instances are launched
AMI	Provides the base image for new instances
EC2 instances	Run the game servers

The Auto Scaling group maintains the EC2 fleet. It can add or remove instances as demand changes and replace unhealthy instances when necessary.

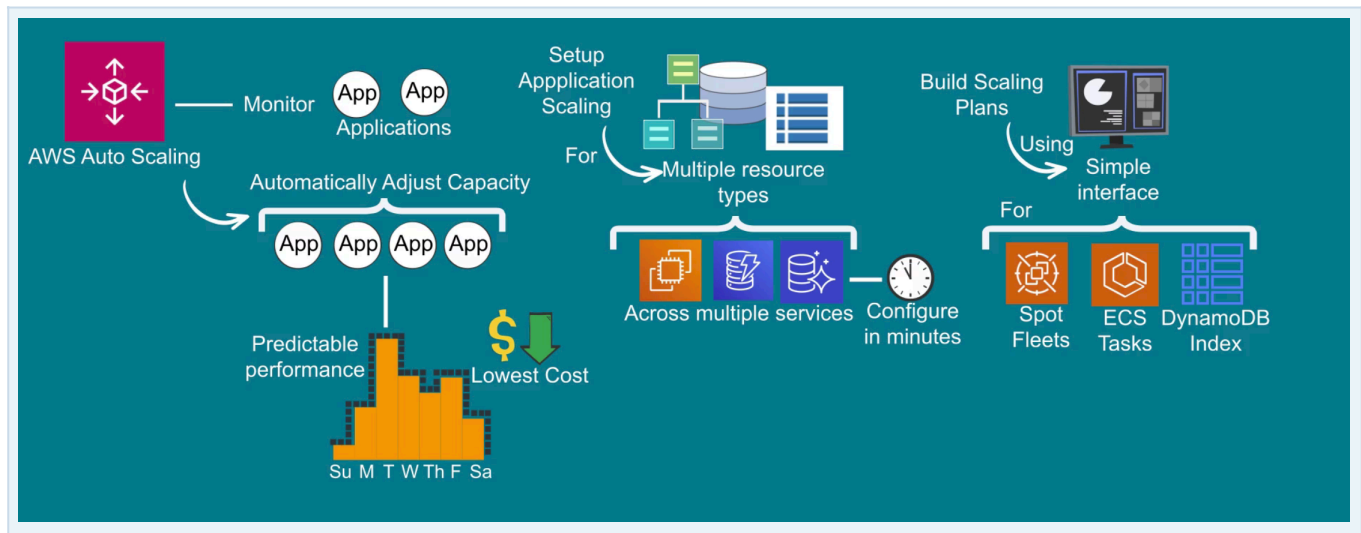
In short

Auto Scaling helps keep the right number of healthy EC2 instances running automatically.

AWS Auto Scaling

How can AWS automatically keep up when demand changes?

Imagine our application has busy and quiet periods throughout the week. Keeping enough resources for peak traffic running all the time would waste money, while having too few could hurt performance. AWS Auto Scaling can monitor demand and automatically adjust capacity.



Feature	Purpose
Monitoring	Watches application demand and metrics such as CPU
Scaling policies	Define when capacity should change
Scale out	Adds capacity when demand increases

Scale in	Removes capacity when demand decreases
Multiple services	Can scale resources across several AWS services

AWS Auto Scaling helps maintain enough capacity for good performance without paying for unnecessary resources. Scaling policies determine when resources should be added or removed based on changing demand.

In short

It adjusts capacity to balance performance and cost.

What does “unhealthy” mean?

An instance is unhealthy when a configured health check determines that it isn’t functioning as expected.

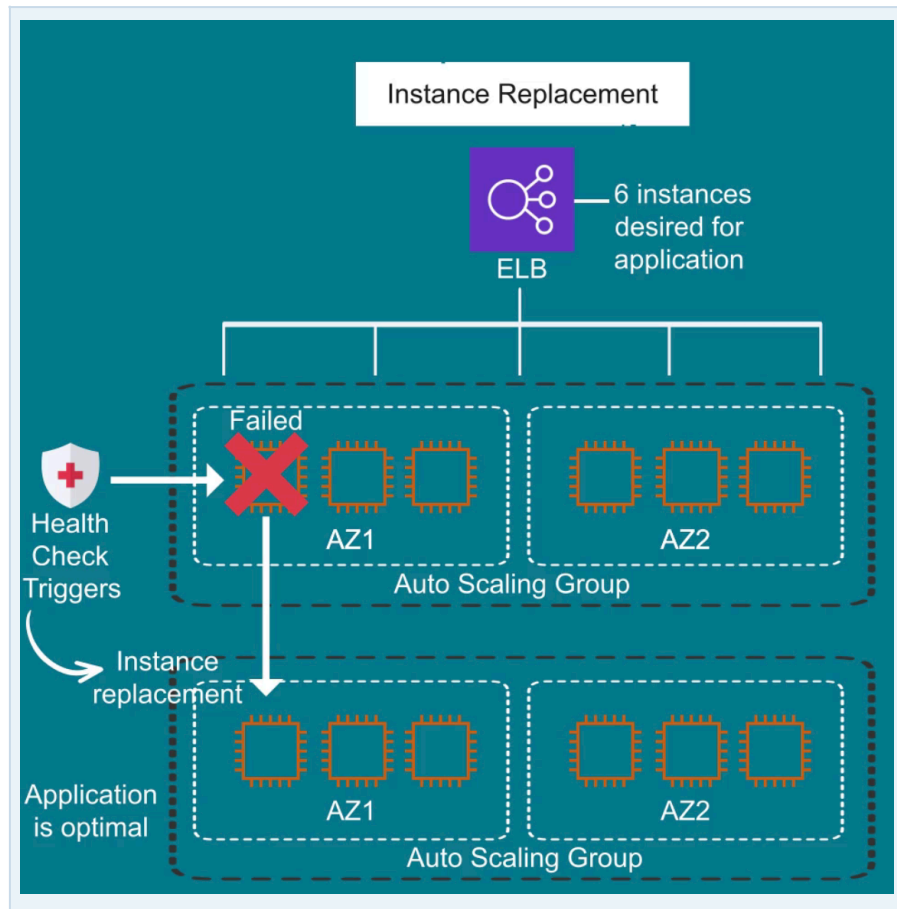
For example, an EC2 instance might crash, become unreachable, or fail a load balancer health check because the application running on it isn’t responding correctly.

A few examples of what might be considered unhealthy:

- Instance failure - the EC2 instance crashes or stops working.
- System failure - AWS detects a problem with the underlying host or network.
- Application not responding - a load balancer sends a health-check request and the application doesn’t respond correctly.
- HTTP error - the application repeatedly responds with an unhealthy status, such as 500 errors.
- Health-check timeout - the server takes too long to respond.
- Custom health check - your own monitoring determines something is wrong and marks the instance unhealthy.

What happens if one EC2 instance fails?

Imagine our application needs six EC2 instances to operate properly, spread across two Availability Zones. If one instance becomes unhealthy, we're suddenly running with only five.



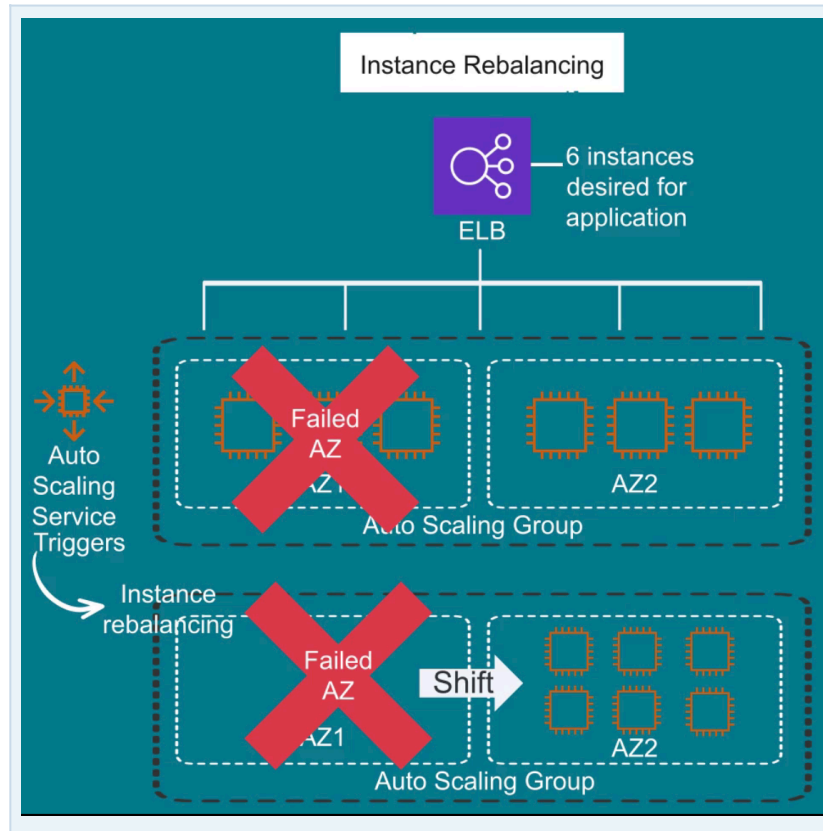
Step	What happens
1. Health check	Detects an unhealthy EC2 instance
2. Replacement	Auto Scaling replaces the failed instance
3. Desired capacity	The group returns to 6 healthy instances

The Auto Scaling group knows that its desired capacity is six. When an instance is considered unhealthy, it can terminate and replace it, restoring the application to its intended capacity.

Auto Healing: Instance Rebalancing

But what if an entire Availability Zone fails?

Now the problem is larger. Instead of losing one EC2 instance, suppose AZ1 becomes unavailable, taking all three of its instances with it.



Before	AZ1	AZ2
Normal	3 instances	3 instances
AZ1 fails	0	3
Rebalanced	0	6

The Auto Scaling group can launch additional instances in the healthy Availability Zone to restore the desired capacity of six. The load balancer can then send traffic to the available instances.

All in all

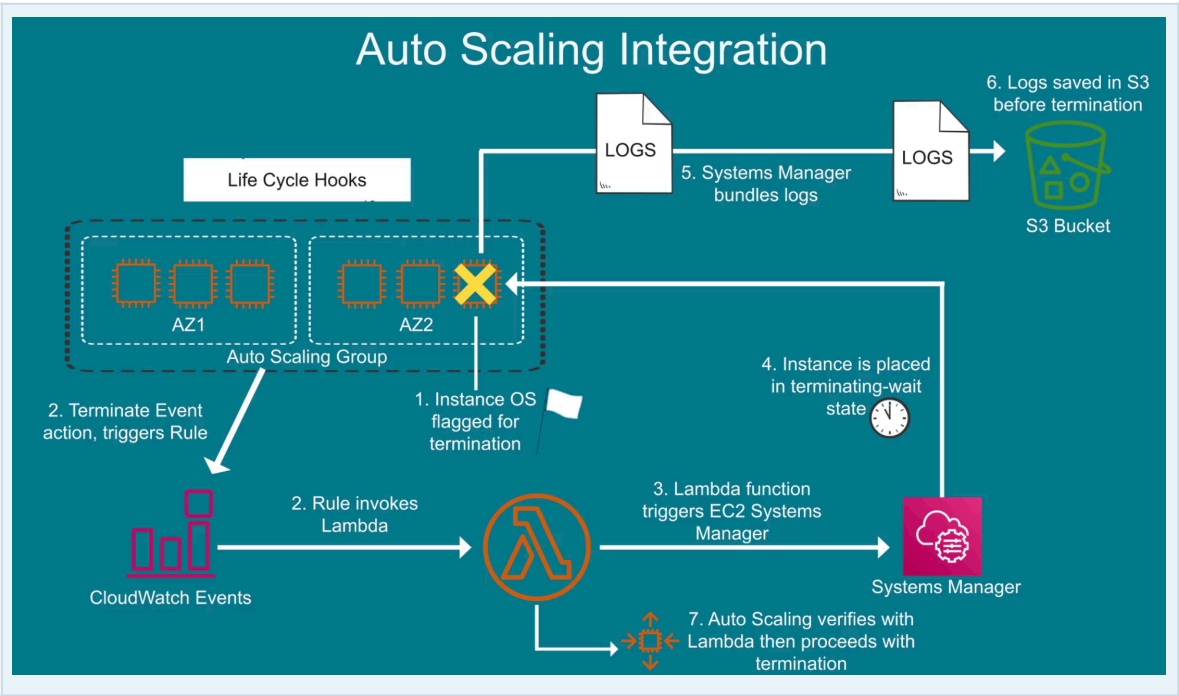
Instance replacement handles individual instance failures, while rebalancing helps maintain capacity when availability across AZs changes.

Lifecycle Hooks

What if we need to do something before Auto Scaling terminates an instance?

Imagine one of our EC2 instances is about to be removed. It might contain logs or other temporary data that we want to preserve before the instance disappears.

A lifecycle hook can temporarily pause the termination process, giving us time to perform those final actions.



Step	What happens
1. Termination starts	Auto Scaling decides to terminate an instance
2. Lifecycle hook	The instance is placed into a waiting state
3. Automation runs	Lambda and Systems Manager perform the required task
4. Save logs	Logs are collected and stored in S3
5. Continue	Auto Scaling is told it can finish terminating the instance

The important part is the waiting state. Instead of immediately destroying the EC2 instance, the lifecycle hook gives other AWS services time to finish necessary work.

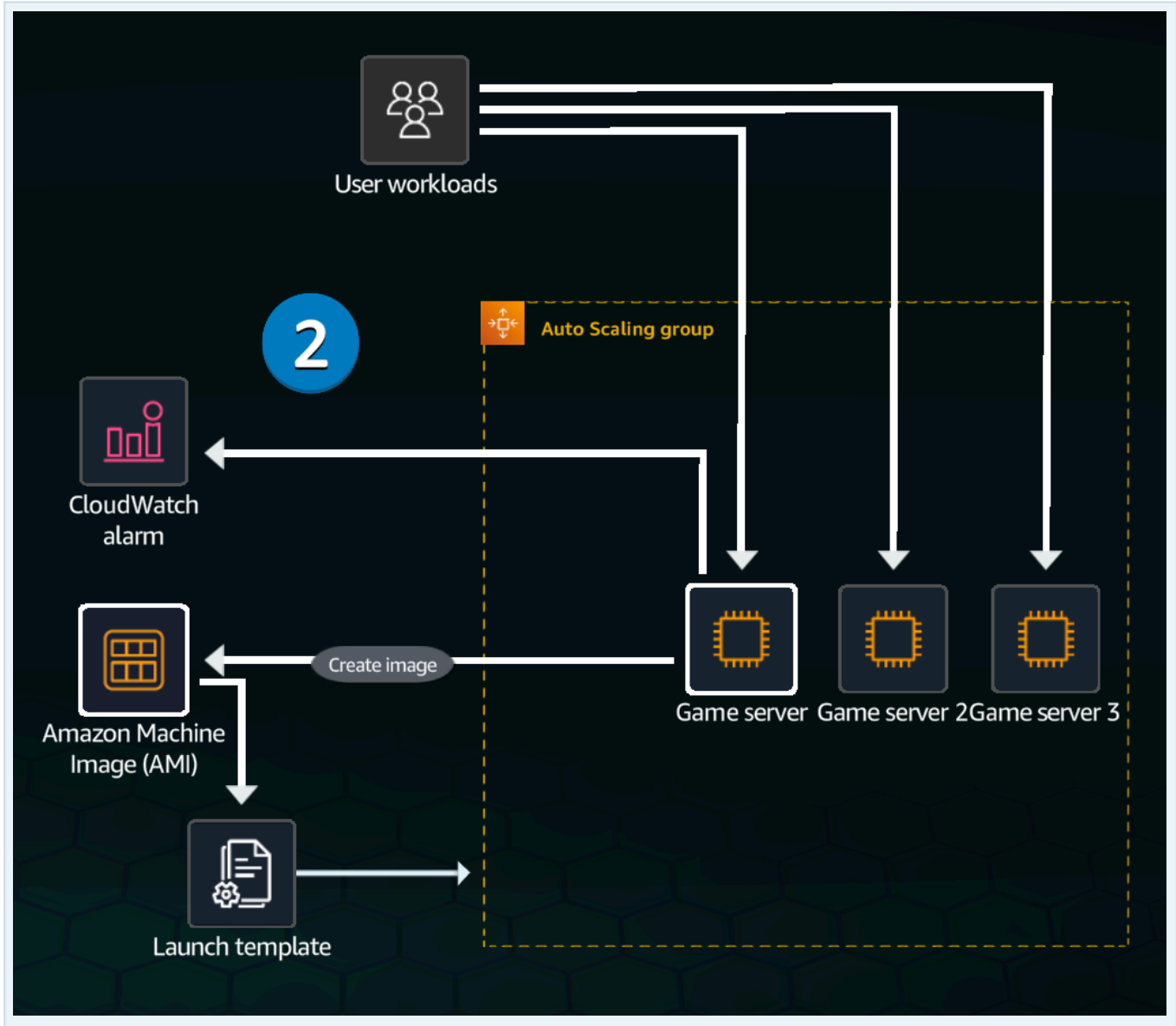
In short
Lifecycle hooks let us pause an Auto Scaling action, perform tasks such as saving logs, and then allow the instance to terminate.

This is normally automated. You can configure what should happen when an instance enters a lifecycle state.

Creating the AMI

So what happens when Auto Scaling needs to launch another game server?

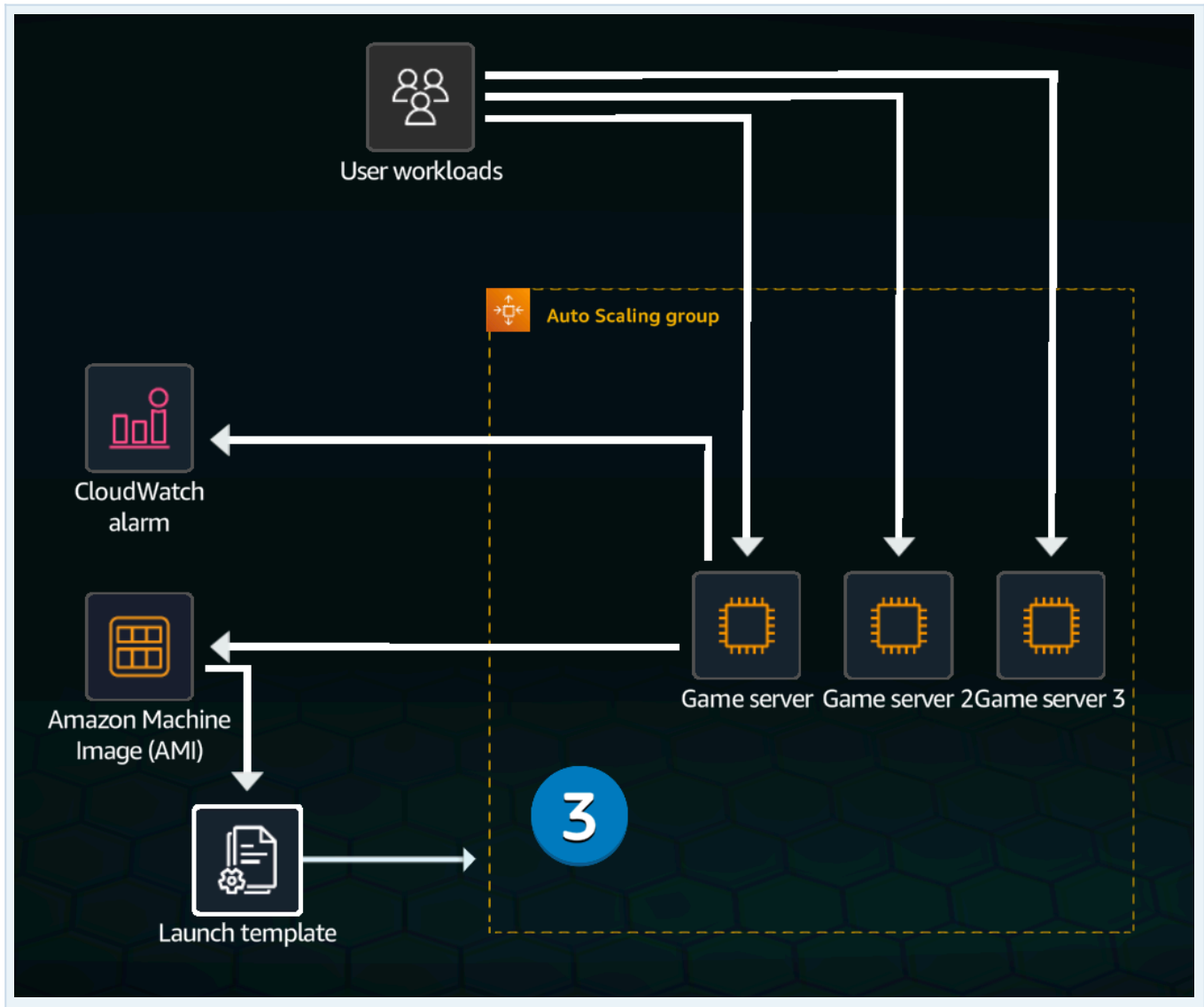
An AMI exists as a reusable copy of our configured application server. The launch template references this AMI, allowing Auto Scaling to launch new EC2 instances with the same application and configuration.



Creating the Launch Template

So how does Auto Scaling know how to create a new EC2 instance?

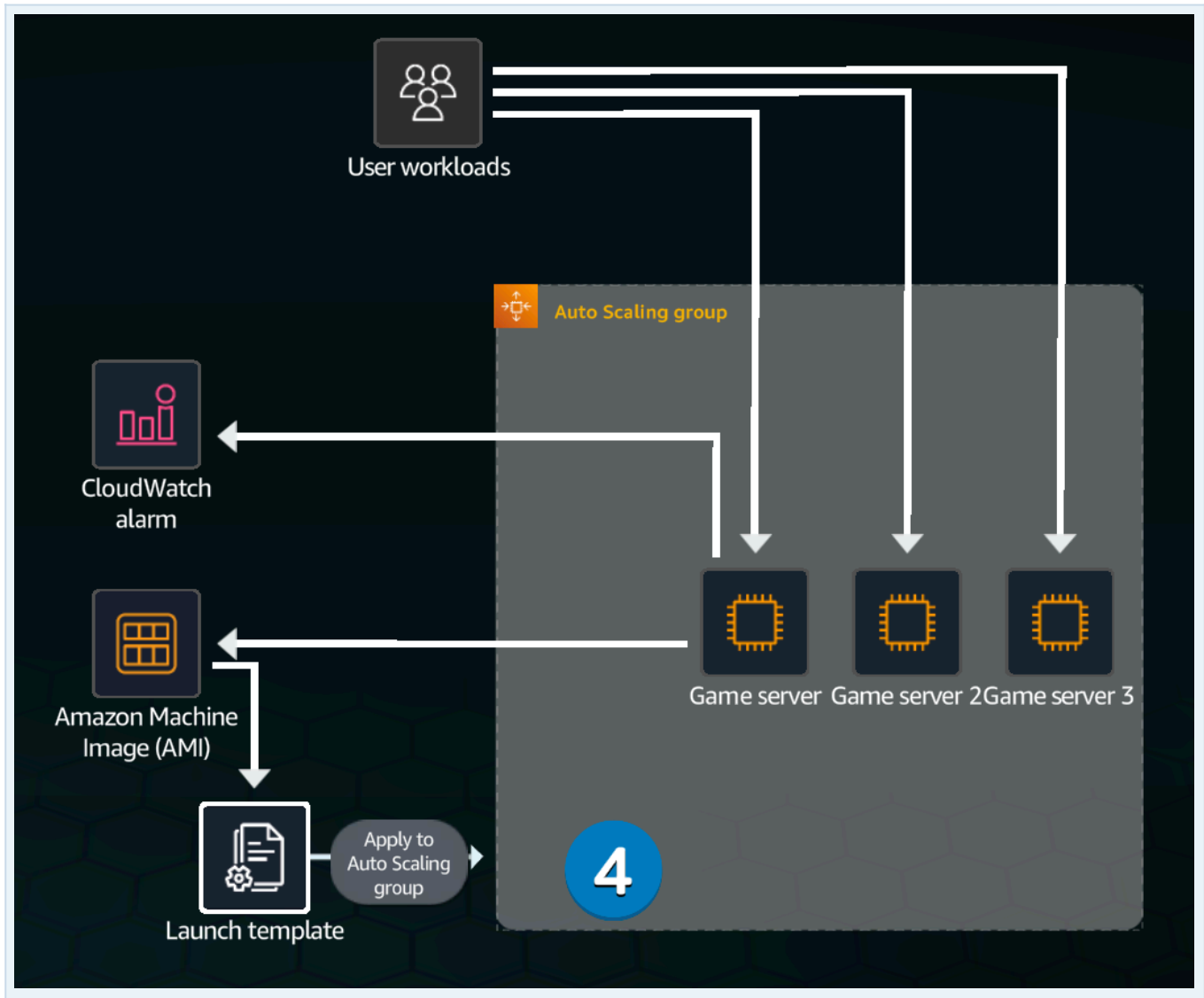
The launch template contains the configuration needed to launch it, such as the AMI, instance type, key pair, and security groups. When Auto Scaling needs another server, it uses this template to create it consistently.



Connect the Launch Template to Auto Scaling

How does Auto Scaling know which blueprint to use?

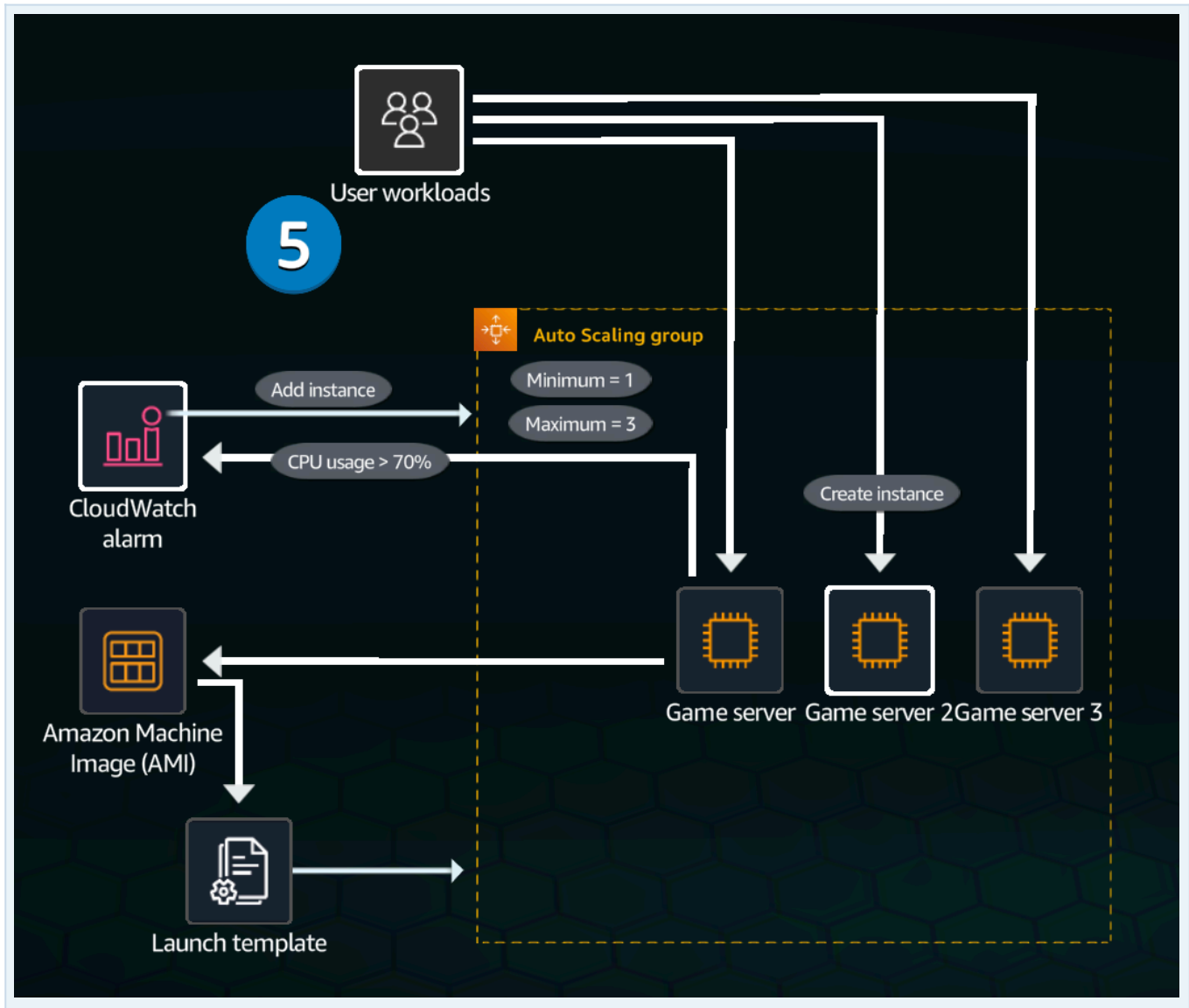
We configure the Auto Scaling group to use our launch template. Now, whenever the group needs to add or replace an EC2 instance, it knows exactly how that instance should be created.



Define When to Scale

So what happens when our existing servers start getting overloaded?

CloudWatch monitors CPU usage, and when it reaches the configured threshold, the Auto Scaling group can add another EC2 instance to spread the workload.



This particular step connects demand to scaling. CloudWatch detects the condition, Auto Scaling decides to add capacity, and the launch template provides the instructions for creating the new server.

This is a good time to start talking about what CloudWatch actually is.

CloudWatch

So how does AWS know what's happening inside our applications and resources?

Amazon CloudWatch is AWS's monitoring and observability service. It collects operational data so we can see how our infrastructure and applications are behaving.

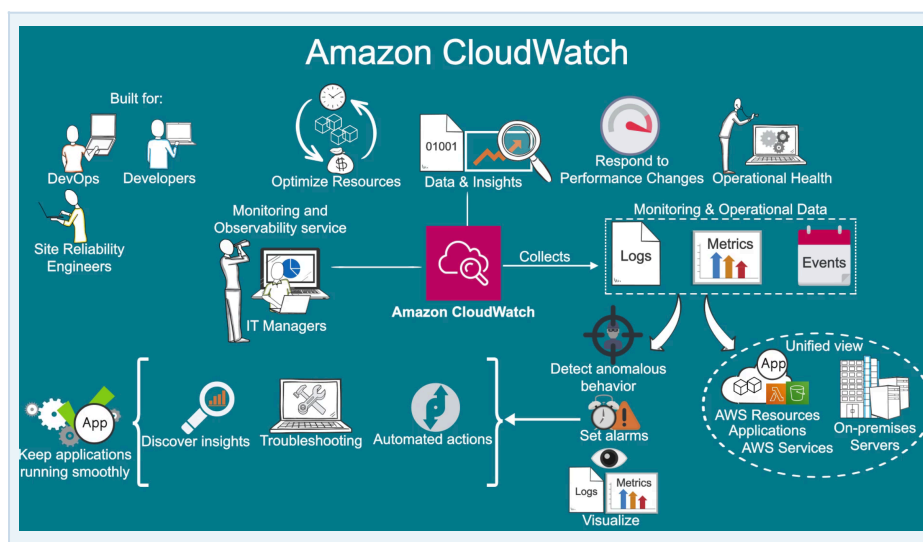
CloudWatch collects	Example
Metrics	CPU usage, request count
Logs	Application and system logs
Events	Changes and activity in AWS resources

CloudWatch can use this information to visualise performance, troubleshoot problems, detect unusual behaviour, and create alarms.

CloudWatch collects	Example	What could happen
Metrics	EC2 CPU reaches 80%	An alarm triggers Auto Scaling to add an instance
Logs	Application repeatedly logs ERROR: Database connection failed	Helps developers find and troubleshoot the problem
Events	An EC2 instance changes state	Can trigger an automated action in response

They serve slightly different purposes: metrics are measurements over time, logs tell us what happened inside a system or app, and events represent something happening or changing.

CloudWatch might show that CPU usage suddenly increased as a metric, application logs might explain that requests are failing, while an EC2 instance state change can be captured as an event that automation can respond to.

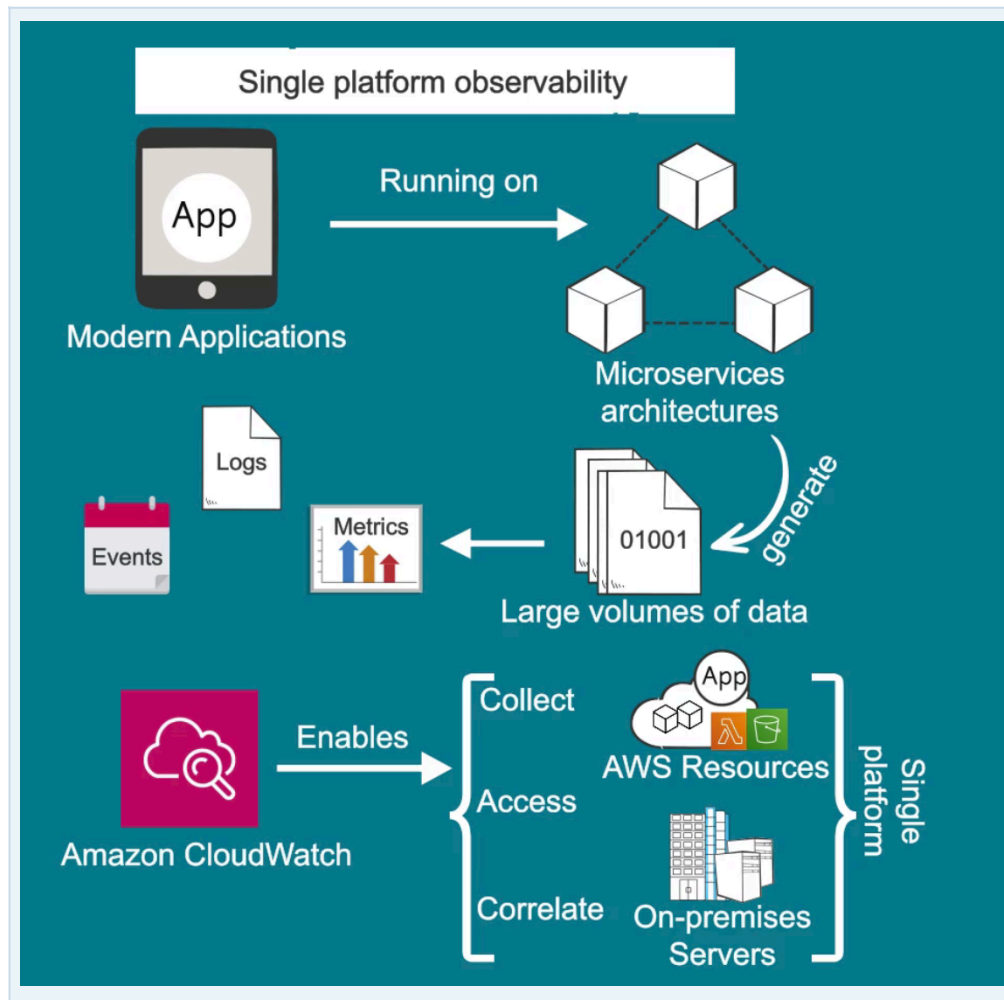


CloudWatch: Key Benefits

Amazon CloudWatch gives us a central place to monitor and understand what is happening across our applications and infrastructure. Instead of checking every AWS service or server individually, CloudWatch brings their operational data together.

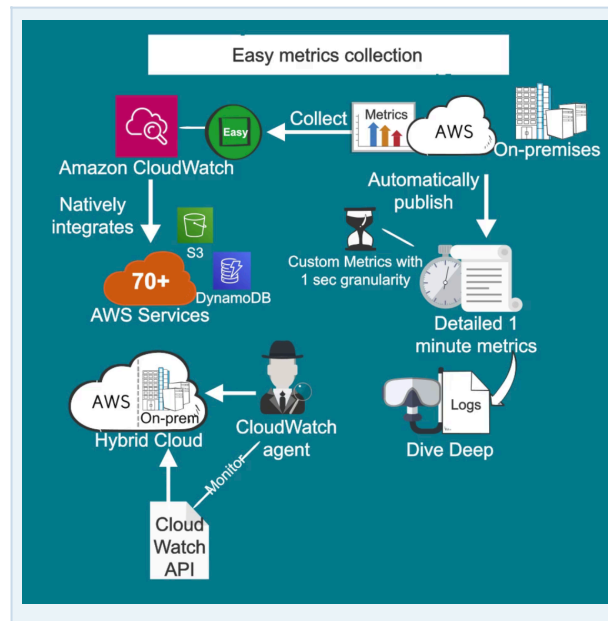
Single-platform Observability

CloudWatch brings metrics, logs, and events from applications, AWS resources, microservices, and even on-premises systems into one place.



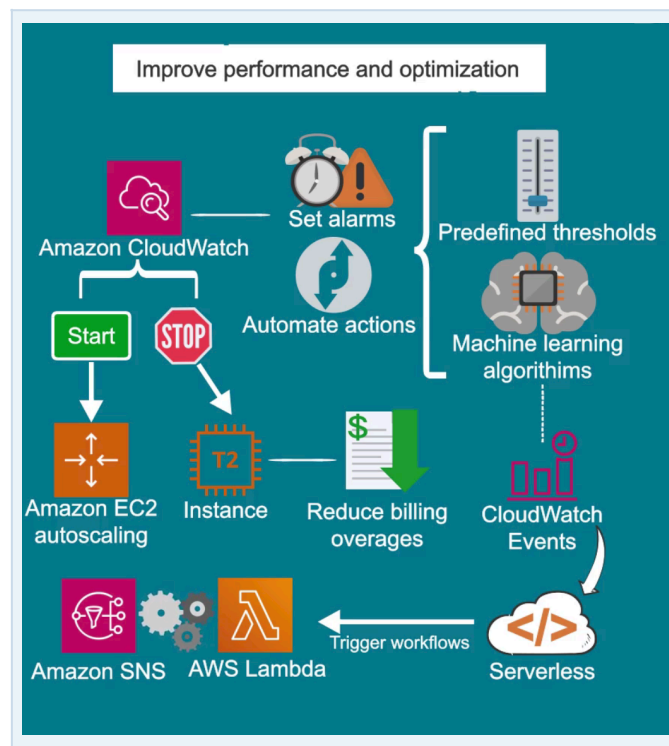
Easy Metrics Collection

Many AWS services automatically publish metrics to CloudWatch. We can also collect additional system data using the CloudWatch Agent or publish our own custom metrics.



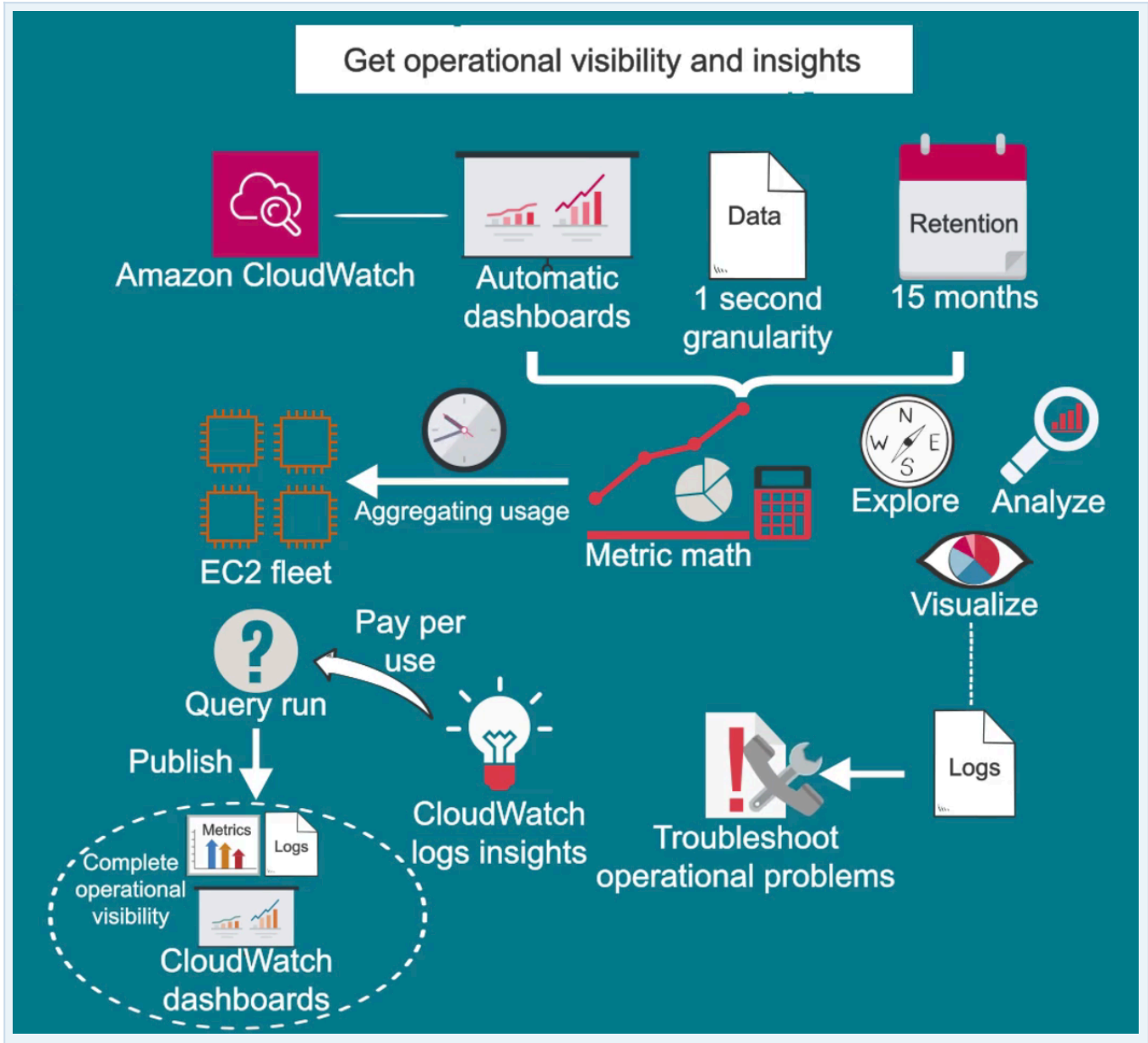
Performance

We can create alarms that react when monitored values reach certain thresholds. These alarms can trigger automated actions, such as Auto Scaling adding EC2 instances when CPU usage becomes too high.



Operational Visibility and Insights

CloudWatch provides dashboards, metric analysis, and log querying so we can investigate problems and understand how our systems behave over time.



Summary

CloudWatch helps us collect, monitor, analyse, and react to operational data, making it easier to keep applications healthy and automatically respond when something changes.

Deciding When to Scale

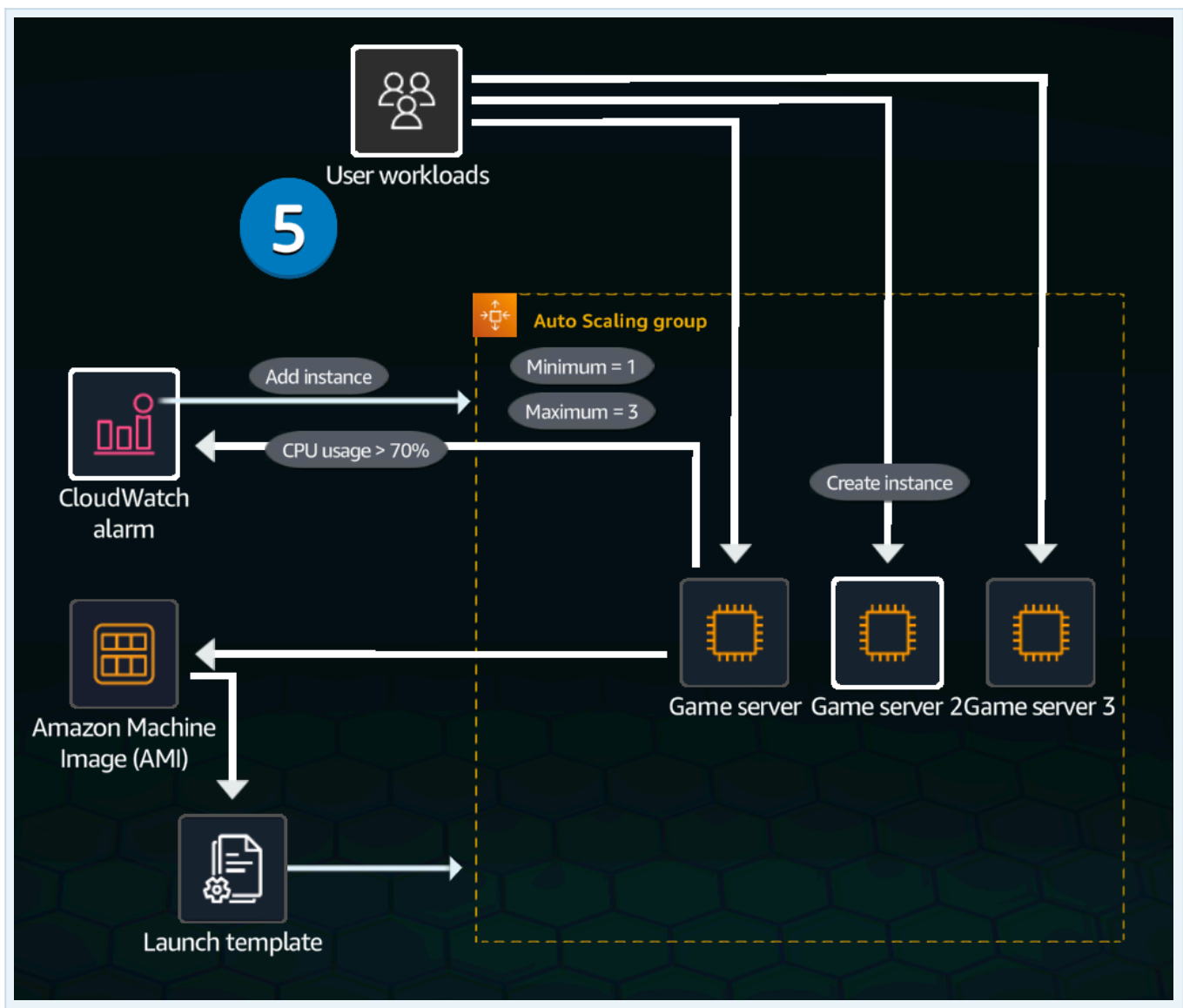
So what happens when our game servers start getting overloaded?

CloudWatch can monitor a metric such as average CPU usage. In this example, an alarm is configured around 70% CPU usage. If the workload pushes CPU usage past that threshold, the alarm can be used by the scaling policy to tell the Auto Scaling group that more capacity is needed.

The Auto Scaling group already knows three important rules:

- Minimum = 1: never scale below one EC2 instance.
- Maximum = 3: never scale beyond three EC2 instances.
- CPU > 70%: indicates that the current instances are under heavy load.

When scaling out is required, the Auto Scaling group uses the launch template we created earlier to launch another EC2 instance with the correct AMI and configuration.



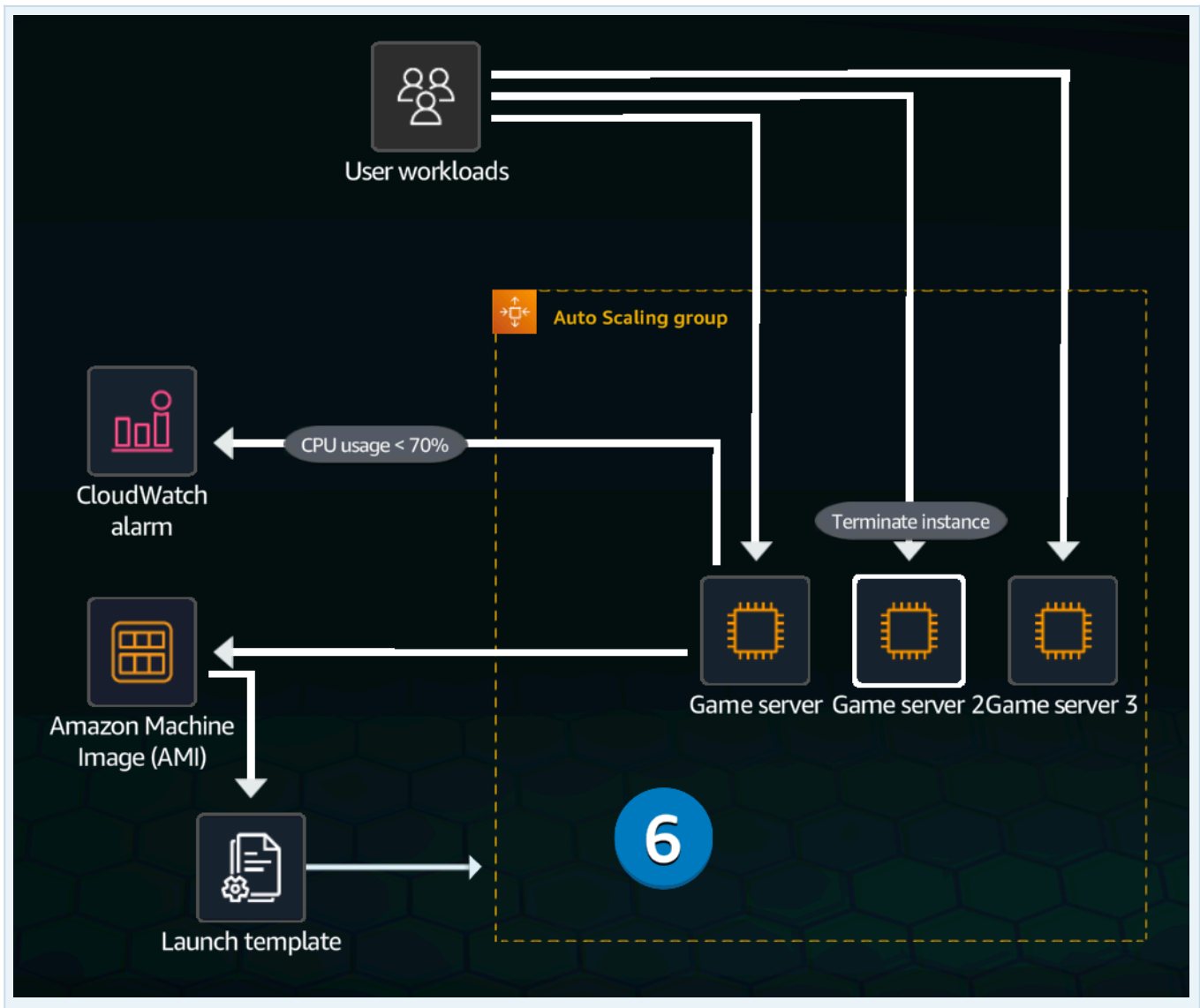
For example, if we're currently running two game servers and their CPU usage becomes too high, Auto Scaling can launch Game Server 3 to help handle the workload. Once three are running, we've reached the configured maximum in this example.

Scaling Back Down

When extra capacity is no longer needed

CloudWatch continues monitoring CPU usage. If the workload decreases and CPU usage remains below the configured threshold for long enough, the scaling policy determines that fewer instances are needed.

Auto Scaling can then terminate one of the EC2 instances, reducing the group's capacity. It will continue doing this when appropriate, but it will never go below the minimum capacity configured.



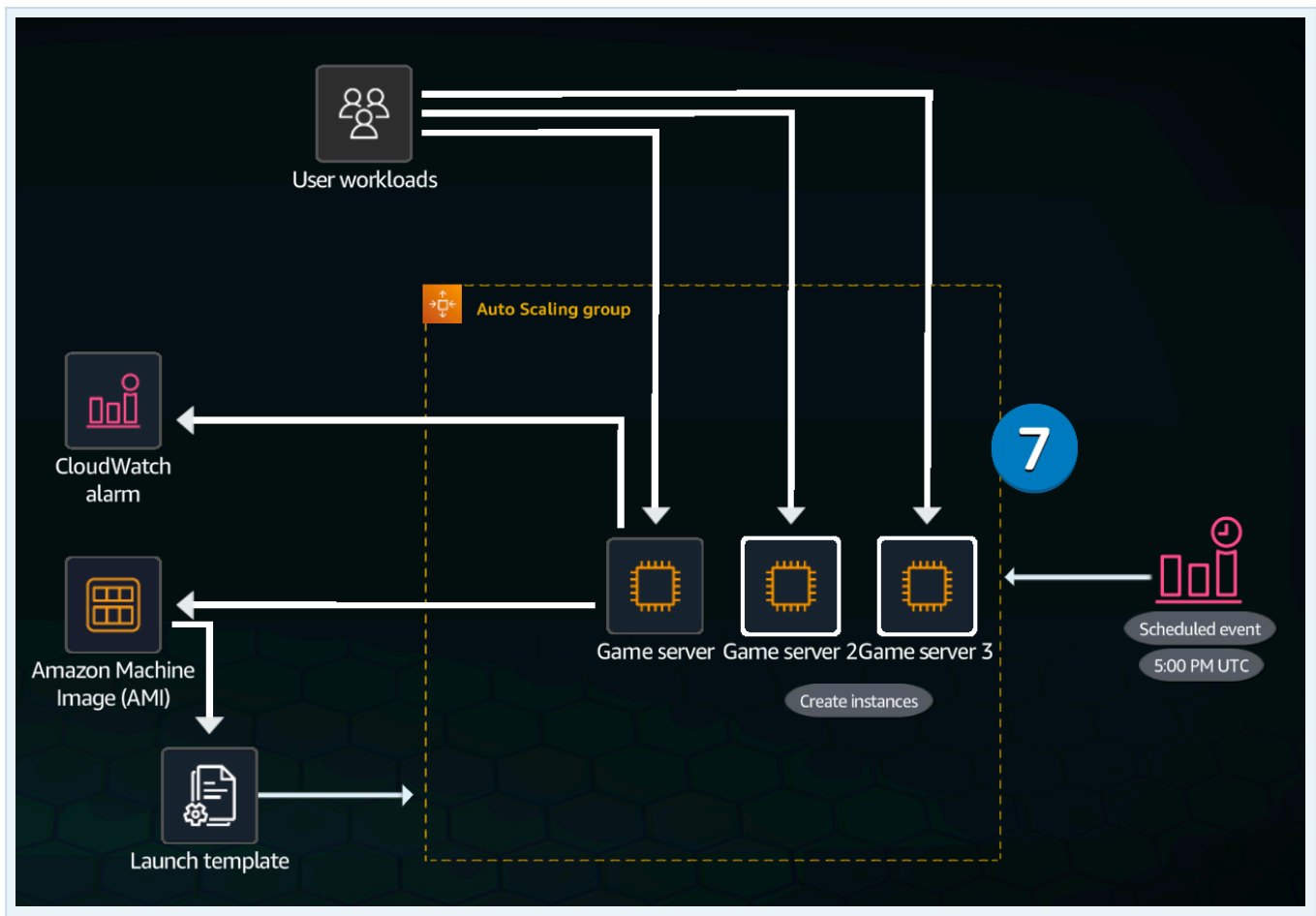
This means that if three game servers are running but the workload can comfortably be handled by two, one of those instances can be terminated.

Scaling on Schedule

What happens if we already know when demand will increase?

We don't have to wait for CPU usage to rise. Auto Scaling can use a scheduled action to change capacity at a specific time.

In this example, we know our game gets busier around 5:00 PM UTC. We can configure Auto Scaling to increase the number of EC2 instances at that time, so the extra game servers are already running before the expected traffic arrives.



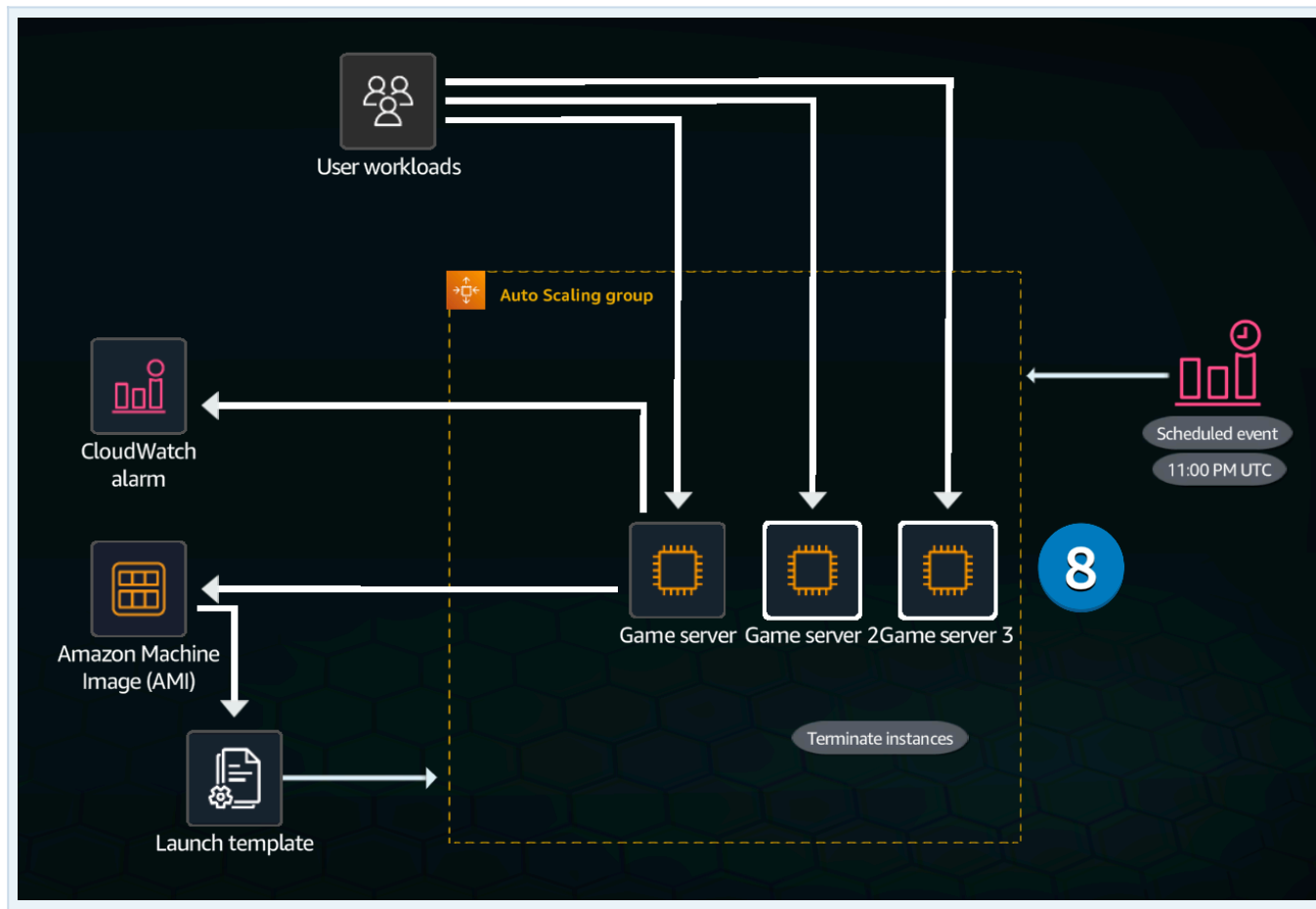
This means users don't have to overwhelm the existing server before Auto Scaling responds.

Scaling Down on Schedule

So what happens when we know the busy period is over?

Just as we scheduled Auto Scaling to add capacity at 5:00 PM, we can create another scheduled action for 11:00 PM, when we expect the workload to drop.

At that time, Auto Scaling reduces the group's desired capacity and terminates the EC2 instances that are no longer needed. In this particular example, the scheduled action shuts down all of them until they're needed again the next day.



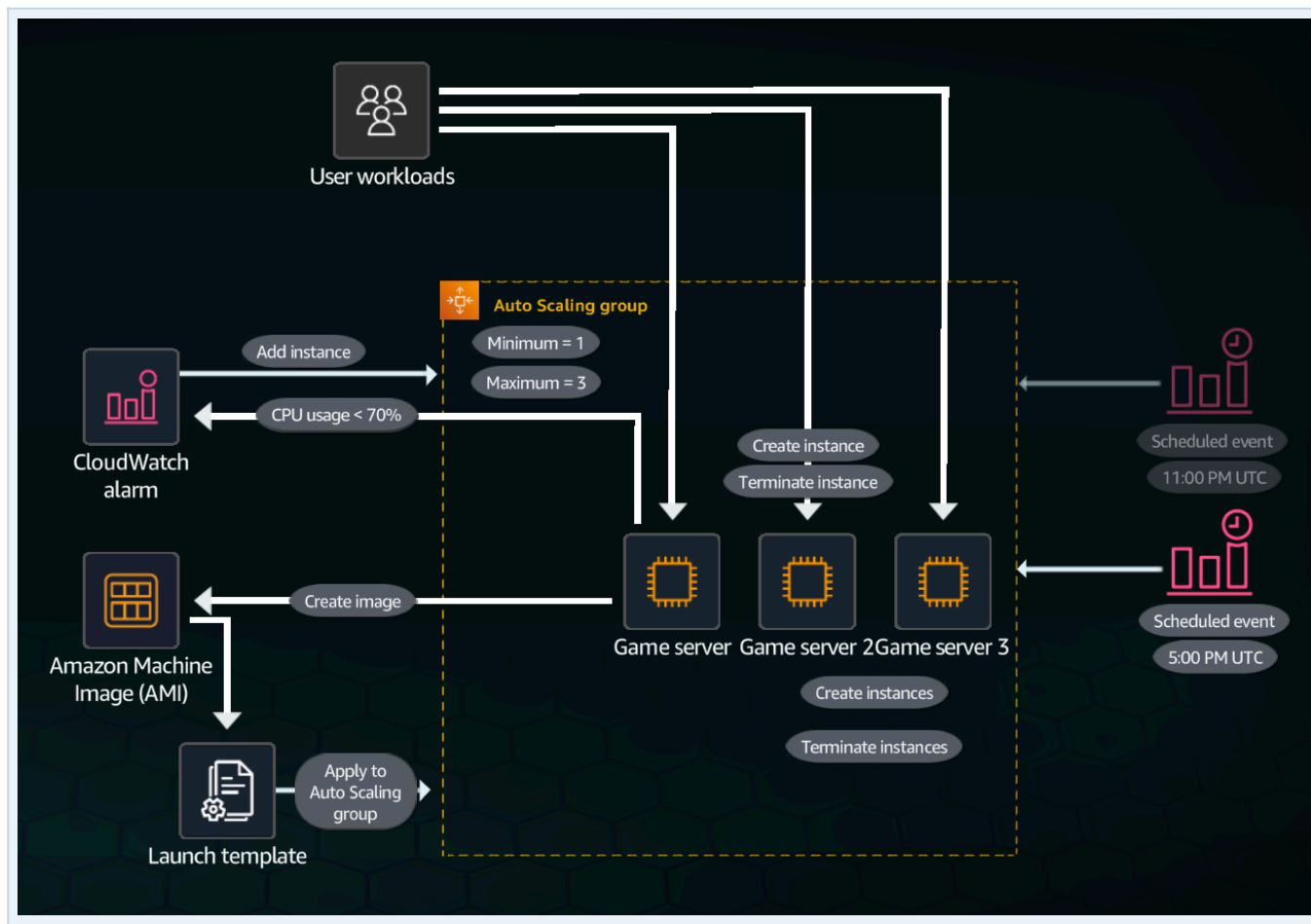
Key point

This doesn't depend on CloudWatch detecting high or low CPU. The time itself triggers the scaling action.

02 Practice Lab

Practice Lab Goals

- Create an Amazon EC2 Auto Scaling group.
- Assign EC2 instances to the Auto Scaling group.



This lab assumes that we already have a configured and running EC2 instance.

Creating a Reusable Image

Navigate to the EC2 instance. Choose Actions > Image and templates > Create image.

Instance summary for i-0fe1410bc2ca29bf2 (Game Server) [Info](#)

Updated less than a minute ago

Instance ID i-0fe1410bc2ca29bf2	Public IPv4 address 3.90.66.250 open address	Private IPv4 address 10.10.0.180
IPv6 address -	Instance state Running	Public DNS
Hostname type IP name: ip-10-10-0-180.ec2.internal	Private IP DNS name (IPv4 only) ip-10-10-0-180.ec2.internal	

Actions

- Instance diagnostics
- Instance settings
- Networking
- Security
- Image and templates**
- Storage
- Monitor and troubleshoot

Create image
Create template from instance
Launch more like this

Name: GameServer

Description: Game server image

Image details

Instance ID
i-0fe1410bc2ca29bf2 (Game Server)

Image name
GameServer
Maximum 127 characters. Can't be modified after creation.

Image description - optional
Game Server Image
Maximum 255 characters

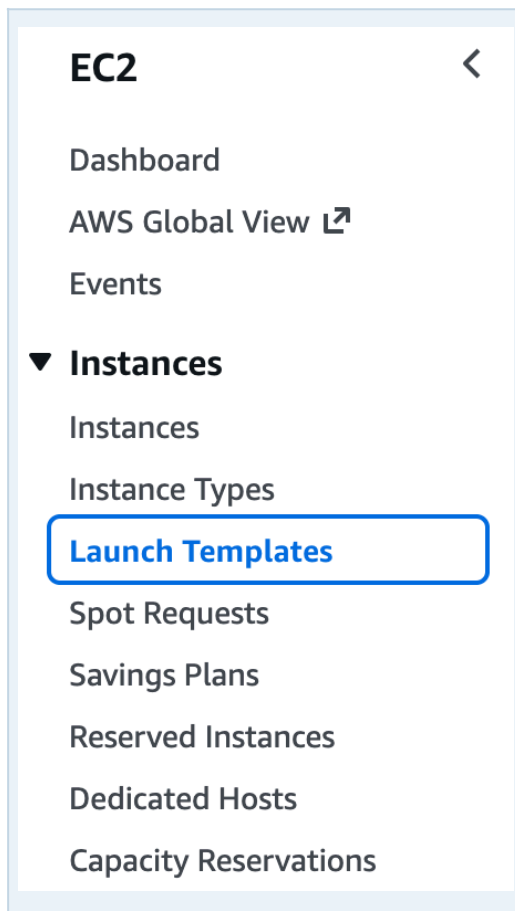
NOTE: “Tag image and snapshots together” applies the same tags to both the AMI and the EBS snapshots created behind it. For example, if the AMI has Name = GameServer and Environment = Production, the snapshots receive those same tags. Keep this option enabled.

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

☒ **Tag image and snapshots together**
Tag the image and the snapshots with the same tag.

☐ **Tag image and snapshots separately**
Tag the image and the snapshots with different tags.

In the left navigation pane, under Images, choose AMIs. Verify that the GameServer AMI shows Available. Then choose Launch Templates and create the template that will use the AMI.



Compute

EC2 launch templates

Streamline, simplify and standardize instance launches

Use launch templates to automate instance launches, simplify permission policies, and enforce best practices across your organization. Save launch parameters in a template that can be used for on-demand launches and with managed services, including EC2 Auto Scaling and EC2 Fleet. Easily update your launch parameters by creating a new launch template version.

New launch template

Create launch template

Creating the Launch Template

Launch template name: GameServerTemplate

Description: Game server template

Keep Auto Scaling guidance checked.

Launch template name and description

Launch template name - *required*

GameServerTemplate

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '*', '@'.

Template version description

Game server template

Max 255 chars

Auto Scaling guidance | [Info](#)

Select this if you intend to use this template with EC2 Auto Scaling

☒ Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

► **Template tags**

► [Source template](#)

Under Application and OS Images, choose My AMIs > Owned by me > GameServer.

RecentsMy AMIsQuick Start

☒ Owned by me

☐ Shared with me


Browse more AMIs
Including AMIs from
AWS, Marketplace and
the Community

Amazon Machine Image (AMI)

GameServer

ami-0014064fa214b8dfe

2026-09-09T09:52:17.000Z Virtualization: hvm ENA enabled: true

Root device type: ebs Boot mode: uefi-preferred

Description

Game Server Image

Architecture	AMI ID
x86_64	ami-0014064fa214b8dfe

NOTE: We already created an AMI from our configured Game Server. The launch template now tells AWS: whenever you launch a new server, use this GameServer AMI as its starting image.

For instance type, choose t3.micro.

▼ **Instance type** [Info](#) | [Get advice](#)

Advanced

Instance type

t3.micro

Free tier eligible

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand Linux base pricing: 0.0104 USD per Hour

On-Demand Windows base pricing: 0.0196 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour

On-Demand SUSE base pricing: 0.0104 USD per Hour

On-Demand RHEL base pricing: 0.0392 USD per Hour

☒ All generations

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

Key pair (login)

Create a new key pair:

- Name: GameServerKeyPair
- Key pair type: RSA
- Private key file format: .pem

NOTE: EC2 instances created by Auto Scaling may still need to be accessed for administration or troubleshooting, so a key pair can be useful depending on the environment.

Create key pair

Key pair name

Key pairs allow you to connect to your instance securely.

GameServerKeyPair

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA
RSA encrypted private and public key pair

☐ ED25519
ED25519 encrypted private and public key pair

Private key file format

☒ .pem
For use with OpenSSH

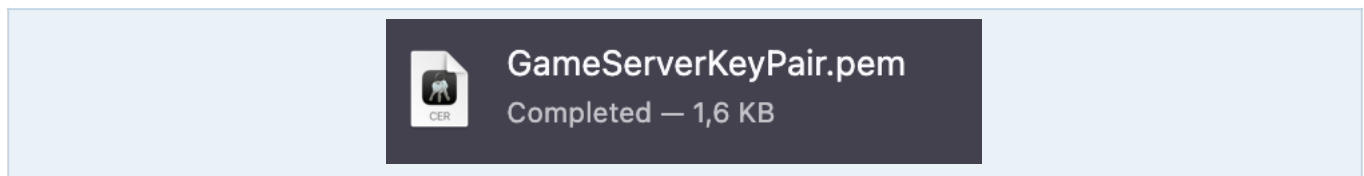
☐ .ppk
For use with PuTTY

 When prompted, store the private key in a secure and accessible location on your computer. **You will need it later to connect to your instance.** [Learn more](#) 

Cancel

Create key pair

Once created, the key pair file should be downloaded.



We won't be doing anything specific with it, this lab, however, it's important for future projects!

Network Settings

Choose Select existing security group and select WebSecurityGroup.

▼ Network settings [Info](#)

Subnet | [Info](#)

Don't include in launch template ▼ [Create new subnet ↗](#)

When you specify a subnet, a network interface is automatically added to your template.

Availability Zone | [Info](#)

Don't include in launch template ▼ [Enable additional zones ↗](#)

Not applicable for EC2 Auto Scaling

Firewall (security groups) | [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Select existing security group ☐ Create security group

Security groups | [Info](#)

Select security groups ▼ [Compare security group rules](#)

WebServerSecurityGroup sg-0e3d4ba4a0cd866 ✕
VPC: vpc-04646de3031835f3a

► Advanced network configuration

NOTE: Leave the subnet unspecified in the launch template. Auto Scaling chooses the subnet later, allowing instances to be placed across multiple Availability Zones.

Review the summary, create the launch template, and choose View launch templates.

▼ Summary

Software Image (AMI)
Game Server Image
ami-0014064fa214b8dfe

Virtual server type (instance type)
t3.micro

Firewall (security group)
WebServerSecurityGroup

Storage (volumes)
1 volume(s) - 20 GiB

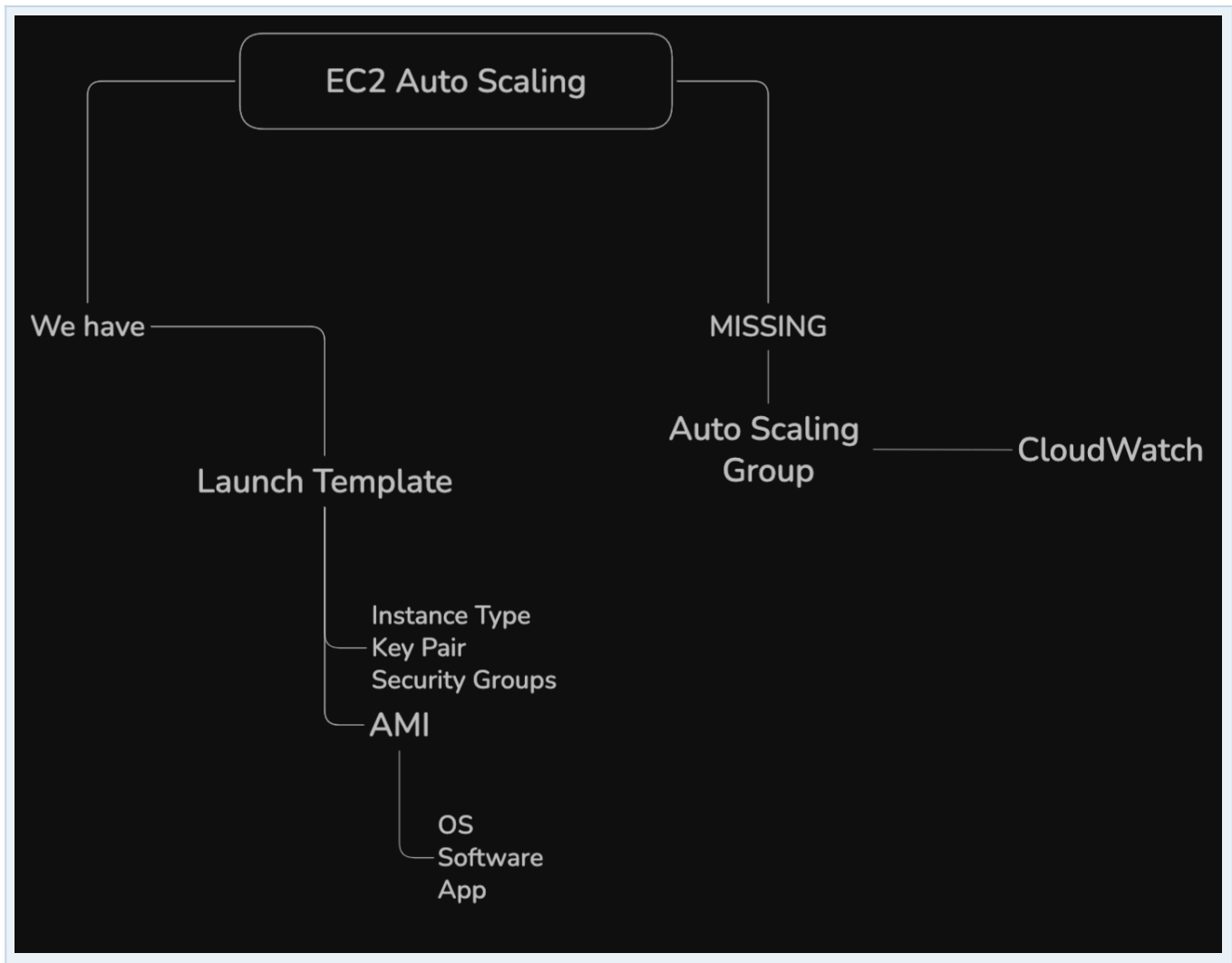
Cancel

Create launch template

By now we have:

- AMI = what goes onto the server: OS, software, and application.
- Launch Template = how an EC2 instance should be created: AMI, instance type, key pair, security groups, etc.

The Auto Scaling group is still missing.



NOTE: Mental model: AMI > Launch Template > Auto Scaling Group > EC2 instances.

Proceed to View Launch Templates and verify that the template was created successfully.

Next Steps

Launch an instance

With On-Demand Instances, you pay for compute capacity by the second (for Linux, with a minimum of 60 seconds) or by the hour (for all other operating systems) with no long-term commitments or upfront payments. Launch an On-Demand Instance from your launch template.

[Launch instance from this template](#)

Create an Auto Scaling group from your template

Amazon EC2 Auto Scaling helps you maintain application availability and allows you to scale your Amazon EC2 capacity up or down automatically according to conditions you define. You can use Auto Scaling to help ensure that you are running your desired number of Amazon EC2 instances during demand spikes to maintain performance and decrease capacity during lulls to reduce costs.

[Create Auto Scaling group](#)

Create Spot Fleet

A Spot Instance is an unused EC2 instance that is available for less than the On-Demand price. Because Spot Instances enable you to request unused EC2 instances at steep discounts, you can lower your Amazon EC2 costs significantly. The hourly price for a Spot Instance (of each instance type in each Availability Zone) is set by Amazon EC2, and adjusted gradually based on the long-term supply of and demand for Spot Instances. Spot instances are well-suited for data-analysis, batch jobs, background processing, and optional tasks.

[Create Spot Fleet](#)

[View launch templates](#)

Launch templates (1) [Info](#)



[Actions](#) ▾

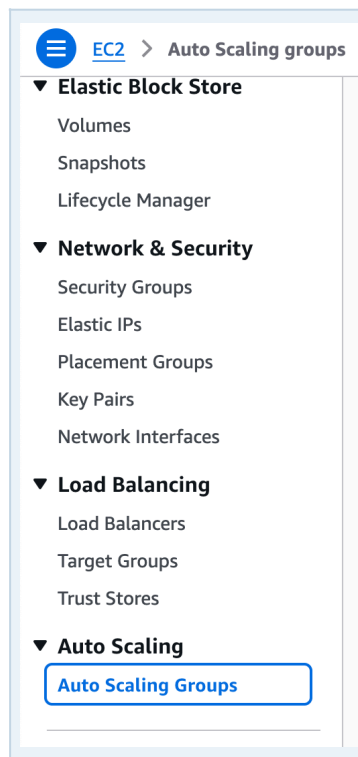
[Create launch template](#)

< 1 >

☐	Launch template ID ▾	Launch template name ▾	Default version ▾	Latest version ▾	Create time ▾	Create time ▾
☐	lt-030608d6da1c055e9	GameServerTemplate	1	1	2026-09-09T10:05:05.000Z	arn:a

Creating the Auto Scaling Group

Navigate to Auto Scaling and choose Create Auto Scaling Group.



Name: RegularCustomerGameServer

Launch template: GameServerTemplate

Name
Auto Scaling group name
Enter a name to identify the group.

Must be unique to this account in the current Region and no more than 255 characters.

Launch template [Info](#) [Switch to launch configuration](#)
Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

[Create a launch template](#)

Under Network, define where the Auto Scaling group is allowed to launch its EC2 instances.

Network [Info](#)
For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.

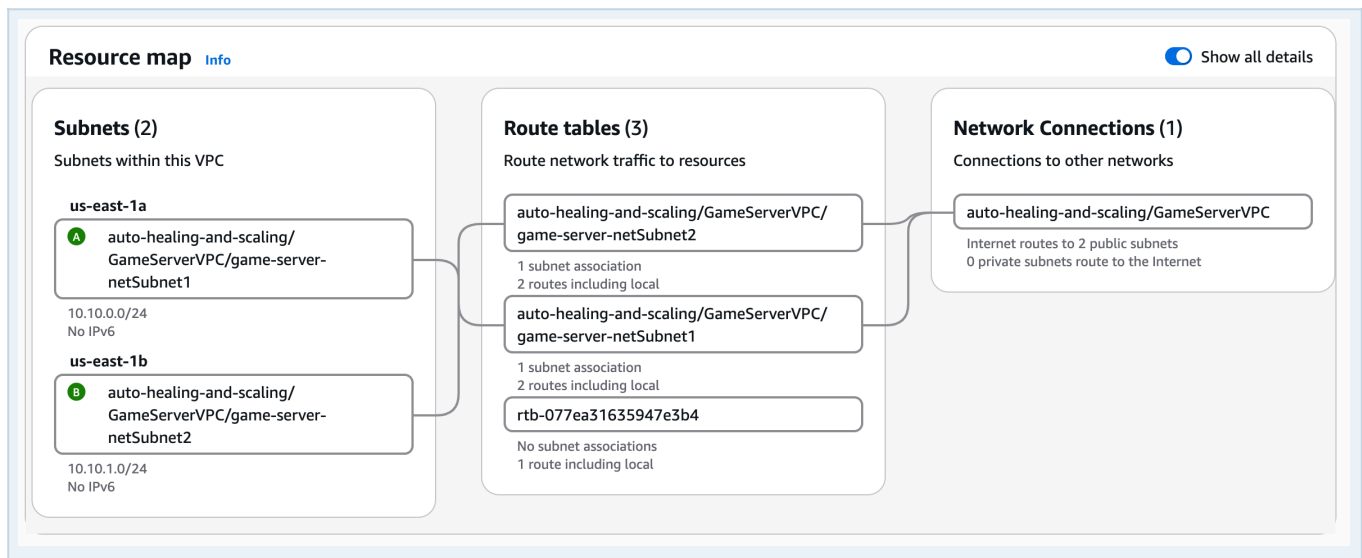
[Create a VPC](#)

Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

[Create a subnet](#)

- Select the VPC the game servers belong to.
- Select which subnets inside the VPC Auto Scaling may use.
- Select subnets in multiple Availability Zones so Auto Scaling can distribute instances for better availability.

NOTE: The VPC was created outside this lab and contains two subnets across two Availability Zones.



We will not use a load balancer or VPC Lattice service in this lab.

Configure the health check grace period to 240 seconds.

Health checks

Health checks increase availability by replacing unhealthy instances. When you use multiple health checks, all are evaluated, and if at least one fails, instance replacement occurs.

EC2 health checks

[Always enabled](#)

Additional health check types - optional [Info](#)

- ☐ Turn on Elastic Load Balancing health checks
Elastic Load Balancing monitors whether instances are available to handle requests. When it reports an unhealthy instance, EC2 Auto Scaling can replace it on its next periodic check.
- ☐ Turn on VPC Lattice health checks
VPC Lattice can monitor whether instances are available to handle requests. If it considers a target as failed a health check, EC2 Auto Scaling replaces it after its next periodic check.
- ☐ Turn on Amazon EBS health checks
EBS monitors whether an instance's root volume or attached volume stalls. When it reports an unhealthy volume, EC2 Auto Scaling can replace the instance on its next periodic health check.

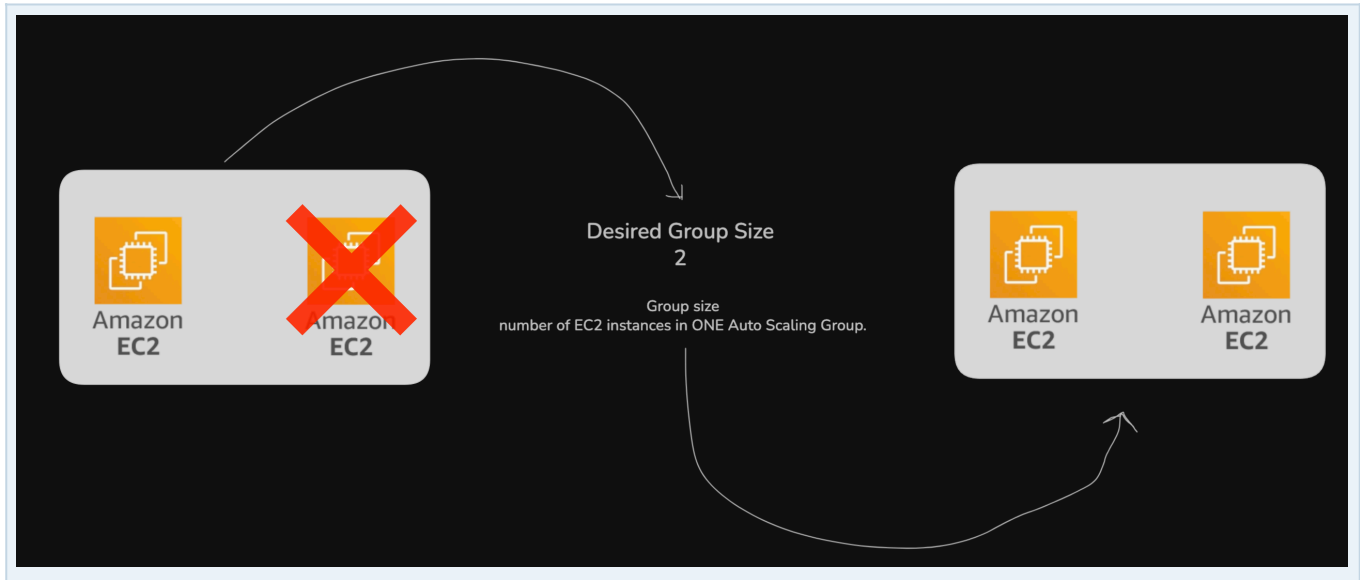
Health check grace period [Info](#)

This time period delays the first health check until your instances finish initializing. It doesn't prevent an instance from terminating when placed into a non-running state.

seconds

Group Size and Scaling Limits

Group size tells the Auto Scaling group how many EC2 instances we want running.



If one EC2 instance fails while desired capacity = 2, the Auto Scaling group works to restore the group to two running instances.

A sensible approach is to start small, monitor real resource usage, define safe limits, and then let Auto Scaling respond to demand. In this lab, desired capacity = 2.

Group size [Info](#)

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type

Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances) ▼

Desired capacity

Specify your group size.

2

Set Minimum capacity = 2 and Maximum capacity = 4.

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

2

Equal or less than desired capacity

Max desired capacity

4

Equal or greater than desired capacity

NOTE: No scaling policies means AWS will not automatically change Desired capacity based on demand.

Instance Maintenance Policy

An important question is what Auto Scaling should do when an EC2 instance needs to be replaced.

Maintenance policy	What happens	Advantage	Trade-off
No policy	AWS uses its default replacement behavior	Simple	Behavior depends on the replacement event
Launch before terminating	New EC2 starts before old EC2 is removed	Better availability	Temporarily costs more
Terminate and launch	Old EC2 is removed while replacement starts	Controls costs	Temporarily less capacity
Custom behavior	Define how far capacity may go above/below desired	More control	More configuration

Instance maintenance policy [Info](#)

Control your Auto Scaling group's availability during instance replacement events. This includes health checks, instance refreshes, maximum instance lifetime features and events that happen automatically to keep your group balanced, called rebalancing events.

Choose a replacement behavior depending on your availability requirements

Mixed behavior

☒ **No policy**

For rebalancing events, new instances will launch before terminating others. For all other events, instances terminate and launch at the same time.

Prioritize availability

☐ **Launch before terminating**

Launch new instances and wait for them to be ready before terminating others. This allows you to go above your desired capacity by a given percentage and may temporarily increase costs.

Control costs

☐ **Terminate and launch**

Terminate and launch instances at the same time. This allows you to go below your desired capacity by a given percentage and may temporarily reduce availability.

Flexible

☐ **Custom behavior**

Set custom values for the minimum and maximum amount of available capacity. This gives you greater flexibility in setting how far below and over your desired capacity EC2 Auto Scaling goes when replacing instances.

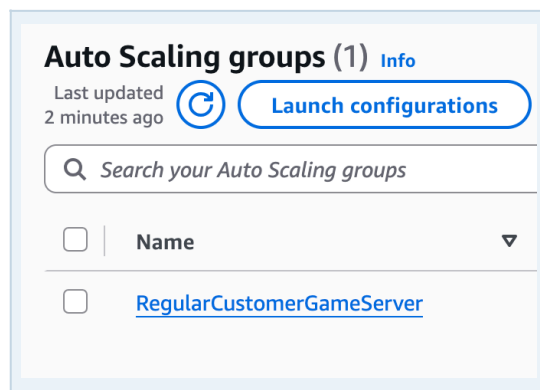
For this lab, leave the maintenance policy unset. Choose Skip to review, review the settings, and create the Auto Scaling group.

Reviewing the Auto Scaling Group

After a scaling policy is created, Amazon EC2 Auto Scaling can monitor the selected metrics. It is useful to review the group's Activity history to understand why the instance count changed.

- Automatic Scaling
- Activity

Open the newly created RegularCustomerGameServer group to find these tabs.



Most sections may still be empty, but Activity should already contain some history.

Activity history (2)			
Filter activity history		Filter by a date and time range	
Status	Description	Cause	
Successful	Launching a new EC2 instance: i-0a0fd8123773e02cf	At 2026-09-09T11:10:50Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-09-09T11:13:51Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	
Successful	Launching a new EC2 instance: i-094ca7fe735f1e480	At 2026-09-09T11:10:50Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-09-09T11:13:51Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	

Creating a Dynamic Scaling Policy

Return to the Automatic Scaling tab and choose Create dynamic scaling policy.

Integrations
Automatic scaling
Instance management
Instance refresh
Activity
Monitoring
Tags

Scaling policies resize your Auto Scaling group to meet changes in demand. With reactive dynamic scaling policies, you can track specific CloudWatch metrics and take action when the CloudWatch alarm threshold is met. Use predictive scaling policies along with dynamic scaling policies in the following situations: when your application demand changes quickly, but with a recurring pattern, or when your EC2 instances require more time to initialize.

Dynamic scaling policies (0)
Info


Actions
Create dynamic scaling policy

Earlier we selected no policies, so we will create one manually:

- Policy type: Target tracking scaling
- Metric: Average CPU utilization
- Target value: 70%
- Instance warmup: 300 seconds

Setting	What it means	In this example
Policy type	How Auto Scaling decides when to change instance count	Target tracking keeps a metric near a chosen target
Scaling policy name	Name of the scaling rule	CPU Utilization
Metric type	What Auto Scaling watches	Average CPU utilization across the group
Target value	Value Auto Scaling tries to maintain	70% CPU; above target can scale out, below may scale in
Instance warmup	Time for a new EC2 to start before metrics fully influence scaling	300 seconds (5 min)
Disable scale in	Prevents removal when demand decreases	Unchecked, so it can add and remove instances

Create dynamic scaling policy

Policy type

Target tracking scaling ▼

Scaling policy name

CPU Utilization

Metric type | [Info](#)

Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.

Average CPU utilization ▼

Target value

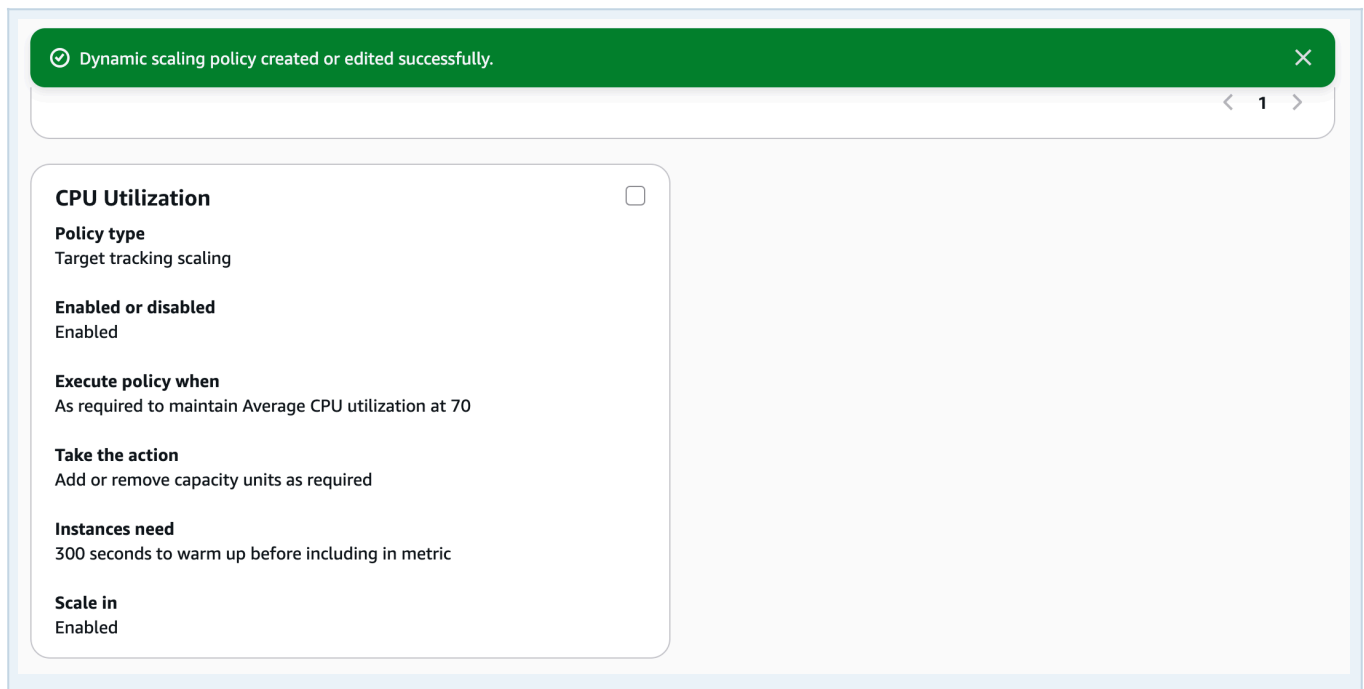
70

Instance warmup | [Info](#)

300 seconds

☐ Disable scale in to create only a scale-out policy

The dynamic scaling policy is now created.



Creating a Scheduled Action

On the same Automatic Scaling tab, create a scheduled action. Here we can set desired capacity based on recurring times and start/end dates.

Name

SecondWaveOfRegulars

Provide at least one value for Desired, Min, or Max Capacity

Desired capacity **Min** **Max**

3 3 4

Recurrence
Schedule a specific date and time for the first scheduled action to run. Interpreted in recurrence time zone: Etc/UTC

Every week (Cron) 0 20 * * Sat

Time zone

Etc/UTC

Current time in selected time zone is 2026-09-09/11:30 UTC

Start date **Start time**

2026/09/12 20:00 Etc/UTC

Format: YYYY/MM/DD. Format: hh:mm.

NOTE: The action “change desired capacity to 3” finishes almost immediately, but Desired remains 3 because that becomes the group’s current setting. It stays there until another scheduled action, scaling policy, or manual change updates it.

NOTE: The End Time means: stop running this recurring scheduled action after this date/time. It does not automatically undo the desired-capacity change.

Return to the EC2 instances overview. Temporary instances running here are expected.

Currently creating AMI ami-0014064fa214b8dfe from instance i-0fe1410bc2ca29bf2. Check that the AMI status is 'Available' before deleting the instance or carrying out other actions related to this AMI.

Instances (3) Info Last updated less than a minute ago

Connect Instance state Actions Launch instances

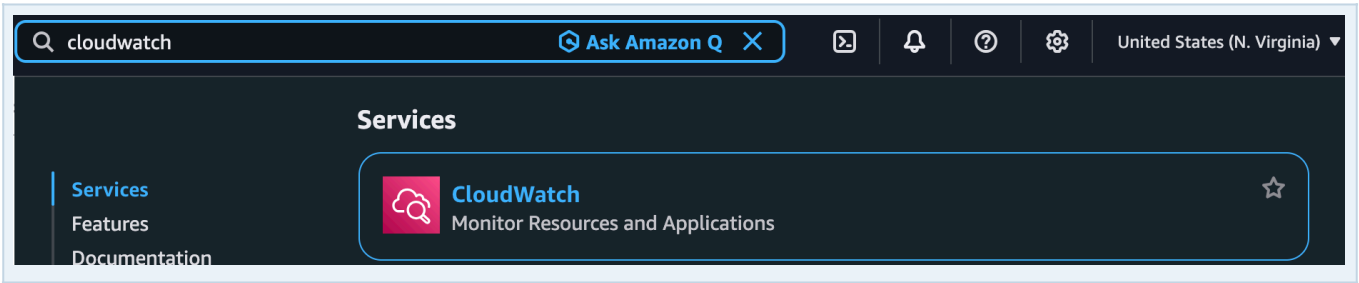
Choose filter set Filter Instance by attribute or tag (case-sensitive)

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	Game Server	i-0fe1410bc2ca29bf2	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-3-90-66-250.cc
☐		i-094ca7fe735f1e480	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-18-205-235-42
☐		i-0a0fd8123773e02cf	Running	t3.micro	3/3 checks passed	us-east-1b	ec2-23-20-152-61.c

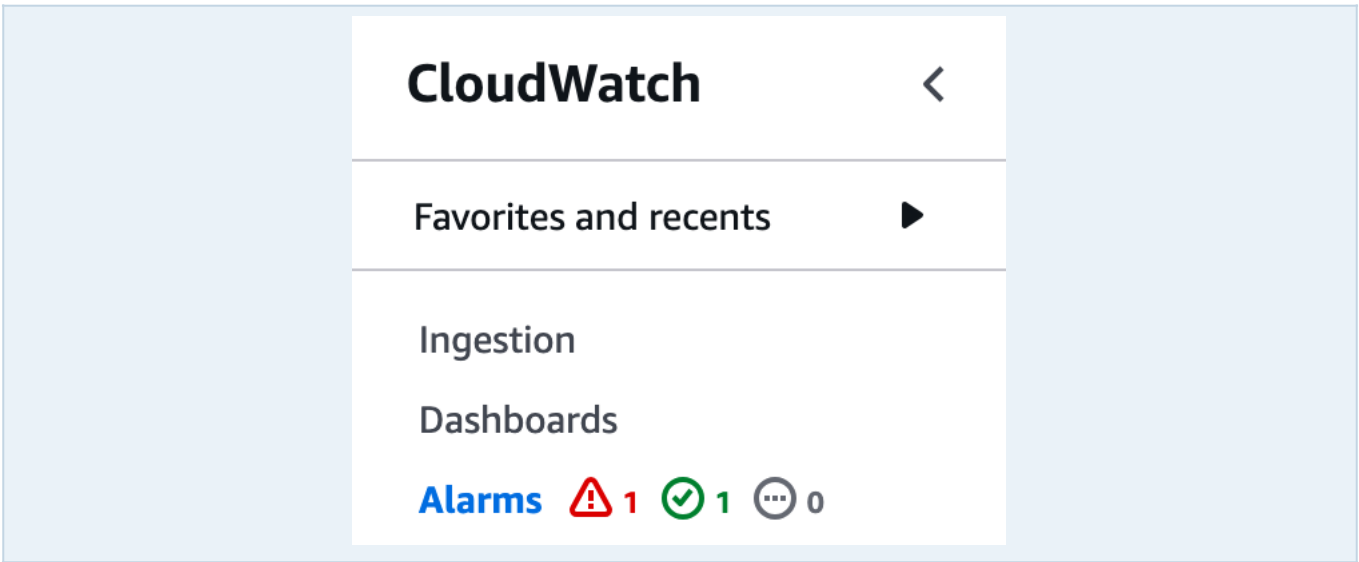
NOTE: If the instances are not displayed, use the section’s refresh button.

Reviewing CloudWatch

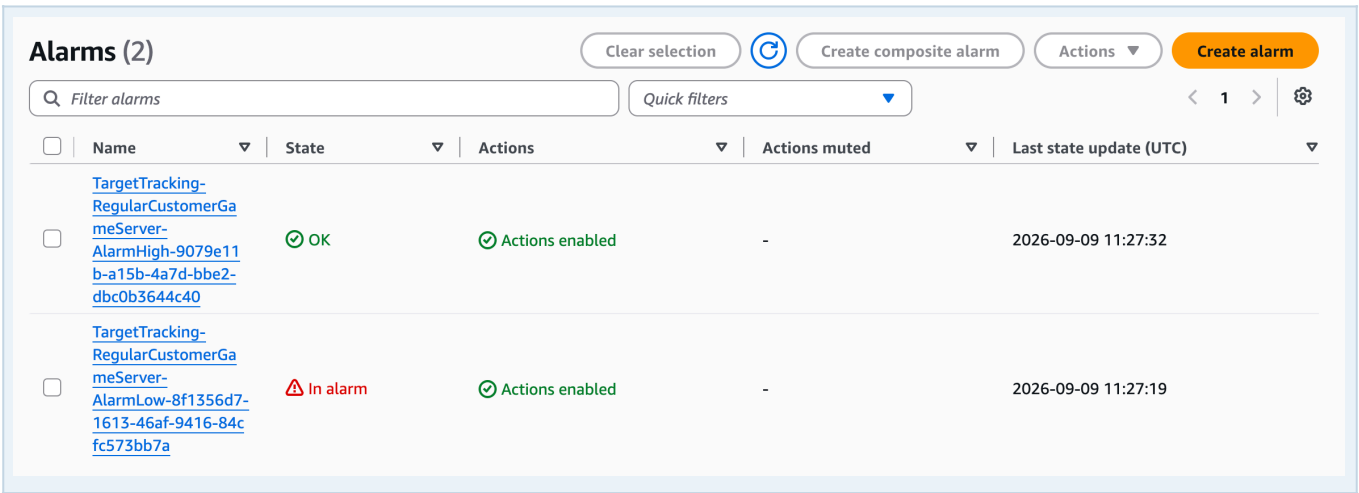
Navigate to CloudWatch.



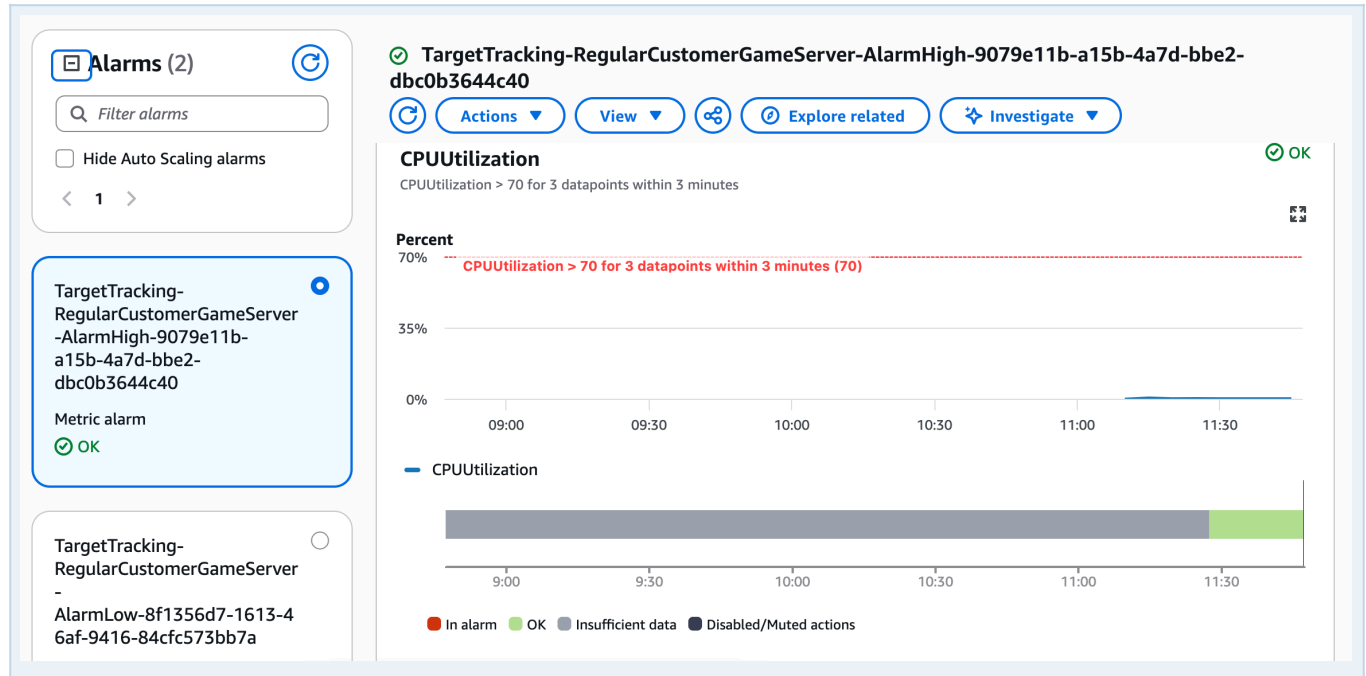
Open Alarms. We should see alarm states that can be inspected.



Open the alarms overview to inspect the additional information.

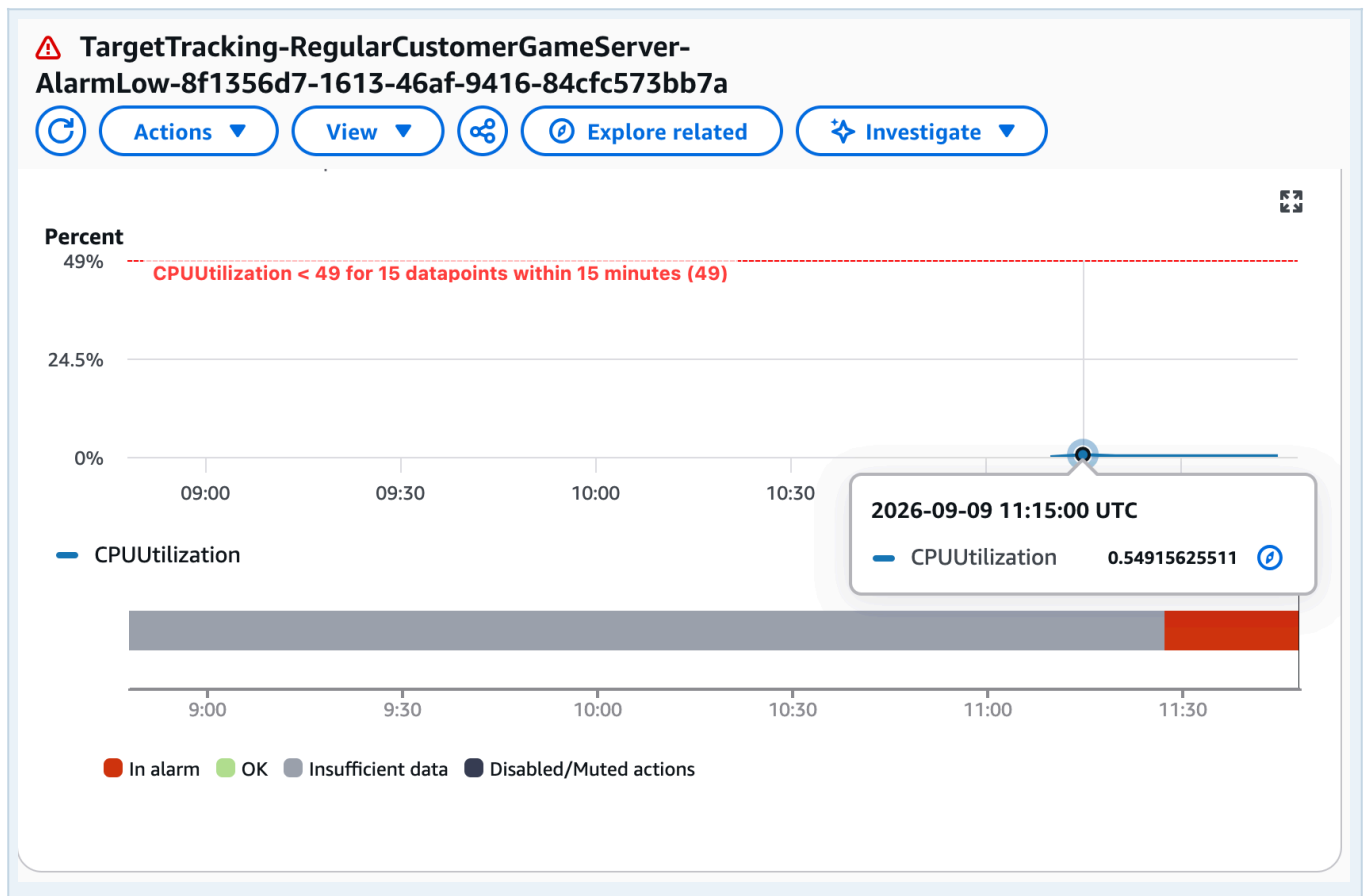


Start with the alarm in the OK state.



Everything looks normal.

Now inspect the other alarm.



NOTE: The blue CPUUtilization line represents the current average CPU usage.

The Actions tab shows the actions associated with the alarm.

Details	Tags	Mute rules	Actions	History	Parent alarms
Actions ✓ Actions enabled					
Type	Description				
Auto Scaling	When alarm transitions to in alarm, use policy CPU Utilization (Maintain the metric ASGAverageCPUUtili				

Scheduling the Daily Shutdown

We are still missing one requirement: at 01:00 AM every day, the company wants the servers shut down to save resources and prevent accidental usage.

Navigate back to EC2 → Auto Scaling Groups → RegularCustomerGameServer → Scheduled Actions and create a new scheduled action.

Name

ServerShutdown|

Provide at least one value for Desired, Min, or Max Capacity

Desired capacity

0

Min

0

Max

0

Recurrence

Schedule a specific date and time for the first scheduled action to run. Interpreted in recurrence time zone: Etc/UTC

Every day ▼

(Cron) 0 1 * * *

Time zone

Etc/UTC ▼

Current time in selected time zone is 2026-09-09/11:52 UTC

Start date

2026/09/09

Format: YYYY/MM/DD.

Start time

01:00

Etc/UTC

Format: hh:mm.

NOTE: This introduces another issue: if desired capacity is set to 0 overnight, it will most likely still be 0 when the stores open at 09:00. A second scheduled action should restore the desired capacity in the morning.

Summary

An Auto Scaling Group (ASG) manages a group of EC2 instances and tries to keep the correct number of healthy instances running.

The **AMI** contains the OS and application. The **Launch Template** defines how new EC2 instances should be created. The **Auto Scaling Group** manages how many instances should be running.

An ASG uses **Minimum, Desired, and Maximum** capacity to control its size.

Auto-healing: If an instance becomes unhealthy, the ASG can replace it to restore the desired capacity.

Dynamic Scaling: CloudWatch monitors metrics such as CPU. High demand can **scale out**; lower demand can **scale in**.

Scheduled scaling: Capacity can also change at predefined times, such as starting extra servers before peak hours or shutting them down overnight.